

电子科技大学
UNIVERSITY OF ELECTRONIC SCIENCE AND TECHNOLOGY OF CHINA

硕士学位论文

MASTER THESIS



论文题目 资源受限下的神经网络模型训练问题

学科专业 计算机科学与技术

学 号 201621060503

作者姓名 叶锦棉

指导教师 徐增林 教授

分类号 _____ 密级 _____

UDC ^{注1} _____

学 位 论 文

资源受限下的神经网络模型训练问题

(题名和副题名)

叶锦棉

(作者姓名)

指导教师

徐增林

教 授

电子科技大学

成 都

(姓名、职称、单位名称)

申请学位级别 硕士 学科专业 计算机科学与技术

提交论文日期 2019.3.23 论文答辩日期 2019.5.22

学位授予单位和日期 电子科技大学 2019 年 6 月

答辩委员会主席 _____

评阅人 _____

注 1：注明《国际十进分类法 UDC》的类号。

The Problem in Training Neural Networks with Limited Resources

A Master Thesis Submitted to

University of Electronic Science and Technology of China

Discipline: **Computer Science and Technology**

Engineering

Author: **Jinmian Ye**

Supervisor: **Prof. Zenglin Xu**

School: **School of Computer Science and Engineering**

摘 要

增加神经网络的宽度和深度是提升模型精度的有效方法，然而计算设备的内存资源成为了限制其发展的主要瓶颈之一。为了缓解该问题，本文提出了两个可行的方案，一是设计出更紧凑的神经网络，降低训练资源消耗；二是减少训练框架本身带来的调度开销，为训练过程提供更多可用的内存资源。

全连接层的低效和冗余是广为人知的，却仍然是循环神经网络（RNN）的基本模块之一。当 RNN 面临高维数据输入时，全连接设计的输入层将成为阻碍模型进一步提升的瓶颈。为此我们对比了低秩矩阵分解和张量分解模型，发现张量分解模型在模型参数量和模型表达能力方面具有优势，于是引入了新的 Block-Term 低秩张量分解模型，提出了紧凑设计的 BT-RNN 模型，以低秩的张量乘法替换了低效的矩阵乘法，相当于用稀疏连接的方式替代了原全连接的设计。对比现有的低秩模型设计如 Tensor-Train RNN（TT-RNN），BT-RNN 不仅在设计上更简洁（超参数的设置更为简单），同时能以更低的参数量获得相同甚至更高的模型精度。在视频分类和图像描述任务中，BT-RNN 在模型精度和收敛速度方面全面超越传统的 RNN 模型和 TT-RNN 模型。尤其是在视频分类任务中，对比 LSTM 模型，BT-LSTM 以极低的参数量获得了超过 15.6% 的精度提升。

为了训练更庞大的网络，常见的做法是以数据并行或模型并行的方式，分别把数据和模型分割并分配到多个计算节点上，但是同时会带来大量的通信开销，严重降低性能。为了提升单节点的模型承载能力，本文提出了一套动态内存调度管理方案，包括：1）张量生存期分析，复用存活期不同的张量内存；2）逻辑统一的内存池实现，以及对应的异步数据传输和 LRU 缓存策略，以极低数据传输开销带来了充足的内存资源；3）代价最优的再计算策略，以少量额外的前向计算开销大幅度降低了网络训练的内存开销。本文所提出的方案能为模型训练过程带来更为充足的内存资源和更快的训练速度，使得探索更宽更深的网络模型成为可能。

关键词：卷积神经网络，循环神经网络，张量分解，内存管理

ABSTRACT

Going deeper and wider in neural networks is an effective way to achieve higher accuracy, while the limited memory resources pose the major bottleneck for improvement. To alleviate this problem, this paper provides two approaches. The first is to design compact neural networks to reduce the model's computation overhead. The second is to reduce the scheduling overhead brought by the training system, offering more available memory for training.

The inefficiency and redundancy of the fully-connected layer are notorious in Convolutional Neural Networks (CNN), but it is still one of the basic building blocks in Recurrent Neural Networks (RNN). The fully-connected input layer in RNN will hinder it from achieving higher accuracy, especially dealing with the high-dimensional input data. We analyzed the low-rank matrix decomposition and the tensor decomposition, and found that the tensor decomposition has advantages in terms of the parameter size and model expressive power. By integrating the Block-Term tensor decomposition with RNN and substituting the low-rank tensor multiplication for the inefficient matrix multiplication, essentially replacing the original fully-connected design with the sparse connection, we finally propose the new compact BT-RNN model. Comparing with the existing low-rank approaches such as Tensor-Train RNN (TT-RNN), BT-RNN is not only more concise (the more flexible hyper-parameters settings) but also able to attain a better accuracy with much fewer parameters. In the video classification task and image captioning task, BT-RNN outperforms the traditional RNN model and TT-RNN model both in model accuracy and convergence speed. Specifically, BT-LSTM achieved an accuracy improvement over 15.6% with fewer parameters in the video classification task comparing with the traditional LSTM.

To train a giant model, the common practice is employing data parallelism or model parallelism and nontrivially dissecting the data or the model across multiple devices respectively, but also incurring excessive communications that drastically deteriorates the performance. To tackle this issue, we present a novel dynamic memory scheduling runtime to enable the network training far beyond the GPU memory capacity, including 3 memory optimizations: 1) *Liveness Analysis*, enabling different tensors to reuse the same physical memory at different time partitions; 2) *Unified Tensor Pool*, being integrated

with the asynchronous data transfer and LRU caching strategy, reserving sufficient memory with a little communication overhead; 3) *Cost-Aware Recomputation*, greatly reducing the memory overhead with a minor amount of extra forward computations. The proposed runtime guarantees sufficient memory while accelerates the training speed, opening new opportunities to explore deeper and wider neural architectures.

Keywords: Convolutional Neural Networks, Recurrent Neural Networks, Tensor Decomposition, Memory Management

目 录

第一章 绪 论	1
1.1 研究背景	1
1.2 神经网络模型精简的背景及研究现状	2
1.2.1 量化和低精度计算	3
1.2.2 知识提取	3
1.2.3 模型裁剪和参数共享	4
1.2.4 低效的全连接层设计	4
1.3 神经网络训练平台内存管理的背景及研究现状	5
1.3.1 模型并行和数据并行	5
1.3.2 细致合理的内存管理	6
1.4 本文的主要贡献与创新	7
1.5 本论文的结构安排	7
第二章 背景知识	8
2.1 循环神经网络	8
2.1.1 原始 RNN 模型	8
2.1.2 LSTM	9
2.2 张量基础	11
2.2.1 张量乘法	11
2.2.2 张量分解	12
2.2.3 CP 分解算法	12
2.2.4 Tucker 分解算法	13
2.2.5 Tensor-Train 链式分解	14
2.2.6 Block-Term 块张量分解	15
2.3 卷积神经网络	15
2.3.1 CNN 模型训练过程	17
2.3.2 设备内存	18
2.3.3 模型的内存观点	19
2.3.4 主流平台的内存管理方案	20
2.4 本章小结	21

第三章 张量循环神经网络	23
3.1 张量化操作	23
3.2 张量分解算法的应用	24
3.2.1 块张量循环神经网络	24
3.3 模型参数量分析	27
3.4 模型复杂度分析	27
3.5 实验	29
3.5.1 视频分类实验	29
3.5.2 图像描述任务实验	31
3.5.3 参数选择实验	34
3.6 本章小结	35
第四章 动态内存管理技术	37
4.1 变量声明	37
4.2 张量生存期分析	37
4.3 统一的内存池技术	40
4.3.1 数据卸载与数据预取	40
4.3.2 Tensor Cache	41
4.4 代价最优再计算策略	41
4.5 动态卷积选择算法	44
4.6 实验	44
4.6.1 单项测试	45
4.6.2 综合测试	49
4.7 本章小结	50
第五章 全文总结与展望	52
5.1 全文总结	52
5.2 未来工作展望	52
致 谢	54
参考文献	55
攻读硕士学位期间取得的成果	65

第一章 绪论

1.1 研究背景

作为一个基础模型，神经网络已经成为机器学习和深度学习领域最重要的工具之一，已被广泛应用于图像识别和理解^[1-5]，语音识别与语音合成^[6]，文本和多语言翻译^[7]，新药物研究^[8,9] 等各行业，推动了整个社会的技术革新浪潮。其中，卷积神经网络（Convolutional Neural Networks, CNN）^[10,11] 和循环神经网络（Recurrent Neural Networks, RNN）^[12,13] 是两个被研究和应用较为广泛的模型。卷积神经网络是一个强大的图像特征提取模型，被广泛应用于图形图像处理领域，如安防领域的人脸识别技术^[14,15]，自动驾驶领域的环境感知^[16,17]，医学领域的医学图像分析^[18,19] 等，很大程度上把人类从繁琐重复的工作中解放。而循环神经网络则擅长处理序列数据，是自然语言处理领域的基础模型之一^[20,21]，同时也被广泛用于推荐系统中的用户行为建模^[22,23]，视频内容分析^[24,25] 等任务。

如今，为了应对与日俱增的海量数据处理需求，一方面需要不断增大模型尺寸（模型参数量和复杂度）以保证推理的精度要求，如 Google 最新的大规模预训练语言模型 BERT^[26]，其参数量已经超过了 3 亿的量级；在推荐系统领域，上十亿、甚至百亿级别参数量的模型比比皆是。庞大的模型在快速消耗硬件计算资源的同时，还降低了模型收敛速度。另一方面需要更强大的计算设备承载模型的训练任务，如 BERT 模型就需要在拥有 64 个 TPU^[27] 的集群中训练超过 4 天时间^①。

现在流行使用高性能的协处理器，如图形计算卡（Graphics Processing Units, GPU）和张量处理器（Tensor Processing Units, TPU）等，完成训练任务。这些协处理器处理速度极快，是中央处理器（Central Processing Unit, CPU）的几十倍；然而另一方面协处理器通常拥有较小的处理内存，目前拥有最大的设备内存的 GPU 是 NVIDIA V100，单卡拥有 32GB 内存^②。然而即使如此，庞大的训练模型仍然无法完整导入到协处理器中^[28]，因此需要扩展使用 CPU 端的内存作为辅助，这带来了频繁的数据搬移和相应的等待开销。这两方面原因导致了协处理器的计算单元无法得到充分利用，计算速度严重受限于片上内存的容量和数据传输速度。虽然现阶段可以通过并行计算和分布式训练缓解单机资源不足的问题，但是同时也带来了严重的通信开销，因此只是资源受限情况下的一个折中方案。

本研究题目从两方面解决内存资源受限情况下的神经网络模型训练问题：1)

① 最新一代的 TPU 训练速度与 NVIDIA V100 速度相当。

② <https://www.nvidia.com/en-us/data-center/tesla-v100/>

从模型构建的角度，设计更为紧凑的模型，提高模型参数共享程度，减少模型参数，降低训练难度；2) 从工程实践的角度，针对现阶段网络训练的特性，设计更合理的 GPU 内存管理方案，减少模型训练过程中的内存压力，提升训练速度。

目前已有众多的模型精简方法，如量化和低精度计算，模型动态裁剪，基于低秩特性的模型静态裁剪方法等，然而针对循环神经网络（RNN）的模型裁剪技术研究较少。本研究题目尝试从模型的静态裁剪方法入手，基于输入数据的高维低秩事实和神经网络参数冗余事实，利用低秩张量分解方法，设计一个更为合理紧凑的 RNN 模型，大幅度减少模型参数的同时，维持甚至提升模型表达能力。另一方面，通过分析目前流行的深度学习框架和卷积神经网络的训练过程，从 GPU 内存管理方面入手，追踪训练过程中产生的中间变量存活周期，精确管理变量的申请和释放时机；同时构建一个设备无关的统一内存池，利用异步数据传输技术隐藏 CPU 和 GPU 内存在逻辑上的差异；此外根据卷积神经网络中部分网络计算简单的特性，通过加入一次额外的前向计算来节省更多的内存空间，从而提速卷积操作的计算，最终达到更快的训练速度。

1.2 神经网络模型精简的背景及研究现状

早期的神经网络由若干个隐藏层组成，每个隐藏层的神经元与下一层的神经元进行全连接，通过反向传播算法进行模型训练^[29]。进一步的研究发现，当数据存在大量的空间局部性，如图像中一个小区域内的像素通常拥有某些共性^[30]，可以通过设计合理的连接方式来减少网络参数，并提高模型性能^[10,11]，这也是卷积神经网络的设计思路之一。

近些年的研究发现，一个足够大的单层神经网络也足以拟合任何想要的模型^[31]，然而此时的模型参数量庞大到即使倾尽全人类的算力也无法完成。同时理论研究发现，深层网络可以以更少的模型参数达到与浅层网络同样的精度^[32]，这一研究发现引领研究人员不断地加深网络，从而满足在计算资源受限时尽可能提高模型性能的需求，AlexNet^[33] 和 VGGNet^[34] 就是两个典型的例子。而后续的 Dropout^[35]，Batch Normalization^[36]，和 Residual Connection^[37] 等模型构建思路的引入则极大地增强了网络的深度和精度，也解决了新模型日益严重的梯度爆炸和梯度消失问题。然而加深网络提升性能的同时，会带来严重的模型复杂度上升的问题，导致训练一个足够好的模型变得越来越困难。一个简单的例子是，利用现代的 GPU，训练一个 AlexNet 模型至少需要 2 天，ResNet^[37] 则需要 4-5 天的时间。加深网络也会导致模型的推理时间变长，难以满足有实时性需求的场景。此外，移动互联网的蓬勃发展，也要求在便携式移动设备上执行模型推理能力，是一个值

得关注的需求^[38]。

然而神经网络的参数冗余性是广为人知的，因此研究人员开始研究通过以下三类手段来简化模型：1) 量化和低精度计算；2) 知识提取；3) 模型裁剪和参数共享，以下对这三类方法做一个简单的调研总结。值得注意的是，本研究题目的关注点主要是第三类方法，即模型裁剪和参数共享。

1.2.1 量化和低精度计算

量化和低精度计算的核心思想是，神经网络模型的数值特征是冗余的。大量的研究表明^[39-42]，神经网络模型可以用更低精度的数值，如 16 位半精度浮点数，16 位定点数，8 位定点数等，甚至是 2 位或 3 位定点数，拟合出接近 32 位浮点数的模型性能。而神经网络中的四大数值：权重 (Weight)，特征图 (Activation)，梯度 (Gradient) 和误差项 (Error)，现在都有了对应的量化方法。由于现代计算机的运算基础是二进制，因此对二值量化的研究较为深入。如在文章 [39,43] 中，利用二进制的连接方法，成功将模型权重压缩到二进制位上，同时为了提升模型的泛化能力而在训练数据中引入了噪音项，也是一个值得学习的方法；在 [42] 中通过将模型权重和特征图量化为二进制数值，同时引入了异或运算提升模型速度，此外引入了一个 scaling factor 变量来提升模型精度。二值量化通常需要引入额外的设计来保持模型精度，此时三值或多值量化则具有精度上的优势。在 [44,45] 中将数值量化到三进制，获得了速度和精度上更好的方案。此外，为了缩减分布式训练过程中的传输开销，同样有针对梯度的压缩和量化方案。文章 [46] 在反向传播时将梯度量化到低精度数值，并加以离散化，只需要消耗极小的内存带宽就能训练足够大的模型；[47] 则是将梯度量化到三进制，使得在并行和分布式训练时获得极大的性能提升。而 [48] 则是量化方面的集大成者，它同时将以上四项量化为 4 位或 8 位定点数值，获得最合适的压缩比和模型性能。此外，量化和低精度计算通常能带来成倍的计算速度提升，然而只能在支持低精度计算的设备上获得加速效果，因此其应用范围受限于边缘计算设备上。量化和低精度计算近年来被广泛研究，尤其是便携式移动设备和嵌入式设备对人工智能能力的需求日益增长，在此类低功耗设备上进行低精度计算，要求模型能以较低的功耗完成计算，同时保持一个合理精度指标。因此，虽然参数表达能力的降低会带来模型精度的损失，但是在实践中一般会采取一个折中方案来平衡模型推理速度、功耗和精度的需求。

1.2.2 知识提取

知识提取 (Knowledge Distillation) 的核心思想是，既然模型参数和结构是冗余的，那么可以利用一个更小更紧凑的模型学习原模型的表达能力。一个简单的

比喻是,原模型是一个知识丰富的教师,教师需要教导一个学生模型;学生模型是一个更紧凑的模型,学生逐步学习老师的知识,追求青出于蓝而胜于蓝,也即在更少参数量、模型更简单的基础上达到甚至超越原模型精度的过程。文章 [49] 首次利用一个更浅层的模型来学习深层模型的表达能力,将知识提取的方法引入了模型压缩的研究领域。[50] 则是利用了模型集成的思想,将多个复杂的深度模型集成,让一个浅层网络学习这些复杂模型的表达能力,从而达到更优的模型精度。[51] 则利用知识提取方法解决了在线蒙特卡洛方法在内存和计算速度上的困难。知识提取是一种新型的训练方法,与其他模型裁剪和压缩方法并不冲突,因此近年来被广泛应用^[52,53]。

1.2.3 模型裁剪和参数共享

模型裁剪和参数共享的核心思想是,通过调整、裁剪神经元的连接方式,达到更优化的参数共享的目的。从方法论而言,主要分为两类方法:1) 动态的模型剪枝,在训练过程中通过分析每一层的统计特征,适当地裁剪神经元;2) 基于参数的低秩特征,静态调整层间连接方式。动态模型剪枝的主要思想是在训练过程中通过分析每一层神经元的权重和输出的统计特征,从而将某些神经元和相应的连接移除。文章 [54] 和 [55] 分别将 group lasso 方法应用于卷积神经网络 (CNN) 和循环神经网络 (RNN) 的动态神经元裁剪上,通过在训练过程中加入 group lasso 惩罚项,迫使同层神经元出现明显的分组现象,然后按组进行裁剪。此外还有类似 [56],在训练过程中设置一个逐步调整的权重阈值,抛弃小于阈值的神经元,也能做到非常好的效果。

静态的模型剪枝思想是,利用权重矩阵的低秩特性,设计全新的层间连接方式,或进行低秩矩阵或张量分解,以达到更好的权重共享目的。从另一个角度看,也就相当于静态改变了层间连接方式,如 CNN 网络就相当于全连接层的一个静态剪枝方法^[11]。文章 [57] 则是通过分析矩阵乘法的性质,引入映射矩阵,以期降低计算复杂度。文章 [58–60] 则是通过引入矩阵分解的方法,将矩阵乘法拆解为若干个小的矩阵乘法过程。近些年的研究表明,对于具有良好局部特征的权重矩阵而言,高维的张量分解^[61]能以更少的参数获得比矩阵分解更好的效果^[62,63],其基本的思想是假设了输入和输出是高维空间的低维流形。

1.2.4 低效的全连接层设计

静态模型剪枝方法作为一种新的模型设计思想,拥有实现容易,不易过拟合,效果良好等特点,已被广泛应用于新的模型设计上。我们观察到,作为一个基本的运算单元,全连接层对于数据的特征捕捉能力低效,且参数极其冗余,相关事实

已经在 CNN 领域被广泛认识和接受^[35,37]，在新型模型的设计上基本被弃用。然而在 RNN 网络结构本身就是由若干全连接层构成，近年来 RNN 网络也常被用于处理图像等高维数据，如视频分类、图像描述等任务，低效的全连接层设计严重制约了其应用的再一步推广。

为了重构低效的全连接层，近年来研究人员引入了低秩张量分解的方法来替代矩阵乘法 $\mathbf{W} \cdot \mathbf{x}$ ，如文章 [62] 最早把 Tensor-Train 分解算法^[64] 引入到了 CNN 领域，替换了网络中低效的全连接层，新模型在计算延迟、模型精度方面有较大优势；文章 [65,66] 则是把 Tensor-Train 算法引入到了 RNN 领域，分别用于处理视频分类任务和超长期序列依赖预测任务上。然而 Tensor-Train 作为一个新型的张量分解算法，由于“信息瓶颈”^[67] 的存在，导致了它的超参数设置较为复杂，而目前在应用方面通常采取了简化的方式，导致了其模型表达能力受限。此外有研究表明^[68]，Tensor-Train 在抗噪音方面性能不佳的问题。

针对上述问题，本文将引入新的 Block-Term (BT) 分解算法^[69]，指导设计出更高效、鲁棒性更好的 RNN 网络结构，同时更好地处理高维数据输入的问题。BT 分解是多个 Tucker 分解的线性组合，在超参数设置、参数表达能力、以及抗噪音方面表现良好，通过合理的参数设置，能获得相比 Tensor-Train 更优的模型精度和更少的模型参数量。

1.3 神经网络训练平台内存管理的背景及研究现状

由于深度学习的理论研究表明，模型的表达能力与模型的宽度和深度正相关。因此，为了追求更高的精度，研究人员把模型做的越来越大。然而精度提高的同时，也带来了训练的困难。如 Inception V4 网络^[70]，拥有超过 515 个基本的计算层，训练过程中至少需要消耗 44.3GB 内存^[28]，然而现在最大的 GPU 片上内存是 32GB，无法满足大型网络的训练需求。计算资源的受限，也会导致研究人员需要分心处理实践中的问题，而不能很好地研究新型网络。一些可行的工程实践能够从不同角度部分解决问题，下面逐一介绍目前可行的方法。

1.3.1 模型并行和数据并行

单机单卡无法满足模型需求时，一个很自然的想法是通过利用多个 GPU 资源共同计算，目前常见的设计思路是单机多卡上的模型并行，和多机多卡上的数据并行^[71]。

模型并行，是将一个完整的模型切割成若干个子模型，如神经网络中可以按层为单元，每一个子模型拥有若干个相邻的层，每一个 GPU 拥有一个独立的子模

型，训练时数据和梯度在不同的 GPU 间传输。然而由于 PCI-e 通道^①的传输速度远远跟不上 GPU 的处理速度，带来了严重的通信开销等待，计算资源并不能得到充分的利用。当年 AlexNet^[33] 在提出时，由于 GPU 的片上内存不够，使用了两块 GPU 做模型并行完成训练，也是无奈之举。

数据并行，也即将数据分块到若干个计算设备上进行处理，最后由一个设备统一处理结果。举例来说，每一个 GPU 拥有一个完整的模型，它们各自处理不同的数据，各自算出最后的损失值和梯度，通过 AllReduce 操作^②同步所有设备上的损失值和梯度值，然后各自更新模型，进行下一轮迭代。目前，数据并行在各大深度学习框架上都有较好的实现方案^[72-76]。然而，数据并行能较好地处理海量数据的情况，但是仍然无法避免模型过大导致 GPU 计算资源得不到充分利用的问题，同时网络传输的速度比 PCI-e 通道更慢，带来了更为严重的通信开销。此外，数据并行意味着每轮迭代处理更大的 batch size，而随机梯度下降算法本身作为一个带偏差的优化算法^[77,78]，更大的 batch size 也同时带来了模型精度的损失^[79]，需要通过工程实践寻找到合理的训练策略。

1.3.2 细致合理的内存管理

模型的训练受限与内存和带宽资源，这个问题可以通过合理的工程实践得到环节。如通过更细致地管理训练过程中的内存申请和销毁，可以缓解设备上的内存压力，从而达到训练更大模型的可能；另一方面通过规划合理的异步数据传输策略，可以减少或避免不必要的 CPU 和协处理器间的数据搬运操作，减少协处理器空转等待的时间。如在 Caffe^[74] 和 Torch^[73] 中，通过复用前向传播和反向传播的内存，可以做到节省三分之一的 GPU 内存消耗。这种策略在传统的线性结构模型能有很好的表现，如 AlexNet、VGG 网络^[34]，然而在非线性结构的模型中难以应用。如 Inception V4，ResNet，这时前向传播和反向传播的变量依赖关系将变得相当复杂，简单地复用策略无法适用。而在 TensorFlow^[72] 和 MxNet^[75] 平台中，则实现了一个基于有向图依赖（DAG）的计算引擎，研究人员只需要显式地定义计算流图，引擎将会自动合并、优化计算过程，并分析每一个节点的存活周期。此外，MxNet 还有一个以层为粒度的再计算策略（Re-computation），它会适当地释放计算量较轻的层（如 ReLU 层），同时引入了额外一次的前向计算过程，来达到减少内存使用的目的。TensorFlow 则充分利用了 CPU 端的内存，通过来回在 CPU 和 GPU 间搬运数据来满足大规模模型的训练。此外，在 CNN 网络中，绝大部分的计算时间集中在卷积层。而目前主流框架的 GPU 计算实现基本上都是利用

① <http://pcisig.com/>

② 现阶段有利用 Parameter-Server 和 AllReduce 两种设计方案，其中 AllReduce 流行更为广泛。

CUDNN 计算库^[80]完成，它实现了多套卷积算法，不同的卷积算法在速度和缓存消耗上各不相同，大致上速度越快的算法需要消耗越多的内存。因此，这也为我们提供了一个加速计算的思路，即通过降低训练过程中的内存消耗，从而提供给卷积层更多的缓存，换取更快的速度。

1.4 本文的主要贡献与创新

本论文以内存资源受限情况下神经网络训练面临的问题为研究重点，从理论研究和工程实践的角度，给出相应的解决方案。本文的主要的创新点如下：

(1) 从模型设计的角度，针对内存资源受限，模型训练效率低下的问题，引入 Block-Term 低秩张量分解算法，用来替换效率低下的 RNN 全连接输入层，同时保持了 RNN 本身的设计思想。在两个重要的任务：视频分类和图像描述任务中表现良好，以极低的参数量获得了远超传统 RNN 模型的精度和收敛速度，同时相比同类方法亦能在同等参数量的前提下获得更优的模型精度，为新型紧凑的网络设计提供了良好的思路；

(2) 从工程实践的角度，针对单机资源受限的情况下大型模型难以训练的问题，参考已有的深度学习训练平台的设计实现方案，通过组合变量生存期分析技术、逻辑统一的内存池实现、Tensor Cache 缓存调度策略和代价最优的再计算策略方案，提出了新的一套动态内存管理方案，并实现了一个新的训练框架 SuperNeurons。实验表明，新的内存管理方案在可训练的模型宽度和深度方面相比现有平台有较大的优势，并且做到了更好的训练速度和所需硬件资源的平衡。

1.5 本论文的结构安排

本文的章节结构安排如下：第一章介绍目前神经网络模型训练时面临的问题及研究现状，即在可用的计算资源受限的情况下进行网络精简化和计算流程的优化。以及针对以上问题本文提出的解决方案。第二章介绍本文提出的新方案所必要的背景理论和最新的计算平台的设计方案，如张量基础和循环神经网络，网络训练的详细过程以及对应的内存观点。第三章详细介绍本文提出的张量循环神经网络的理论、模型构建过程、训练细节和模型复杂度分析，并在实验中展示其在两个重要的任务中的表现。第四章详细介绍本文提出的动态内存管理技术，重点介绍三个内存优化技术，并以此构建的一整套管理方案。第五章总结全文，并分析本文从模型设计和工程实践两个角度提出的方案，说明其存在的不足，以及未来可能的优化方案。

第二章 背景知识

在介绍本文提出的张量循环神经网络和动态内存管理技术方面的细节前，先介绍其所必须的背景知识：（1）循环神经网络的设计思想和其关键变种，所面临的关键问题；（2）引入张量的概念和张量分解算法，阐述其在低秩分解和模型压缩方面的重要作用；（3）神经网络训练过程和细节，以及从内存管理的角度分析现有平台实现方案的优缺点。

2.1 循环神经网络

循环神经网络最初由 Rumelhart 等人于 1986 年提出^[12,13]，是一个用于序列建模的网络模型，被广泛应用于自然语言处理领域、视频内容分析、用户动作序列预测、天气预测和股市预测等任务。现在最为广泛流行的两种网络结构分别是 LSTM（Long Short-Term Memory）^[81] 和 GRU（Gated Recurrent Unit）^[82]，两者在不同任务上表现各有优劣，为了行文方便，本文仅以 LSTM 为例，相关理论和技术同样适用于 GRU。

2.1.1 原始 RNN 模型

循环神经网络（RNN）顾名思义，其内部的网络结构呈现出“循环，递归”的结构。其核心思想是把序列中的“记忆”内容通过计算后存储到网络的一个隐层向量 $\mathbf{h}$ 中，每处理一个序列元素时会根据已有记忆计算并更新模型，形成新的“记忆”。数据以序列的形式输入，并根据上一时刻的隐层状态 $\mathbf{h}_{t-1}$ 与当前输入 $\mathbf{x}_t$ 计算得到新的隐层状态 $\mathbf{h}_t$ ，其结构可见图2-1，计算过程见公式 (2-1) 和公式 (2-2)：

$$\mathbf{h}_t = \sigma_h(\mathbf{W} \cdot \mathbf{x}_t + \mathbf{U} \cdot \mathbf{h}_{t-1} + \mathbf{b}_h) \quad (2-1)$$

$$\mathbf{o}_t = \sigma_o(\mathbf{V} \cdot \mathbf{h}_t + \mathbf{b}_o) \quad (2-2)$$

其中， σ_* 表示激活函数，通常使用 sigmoid 函数^①或 tanh 双曲正切函数等； $\mathbf{x}_t \in \mathbb{R}^I$ 表示 t 时刻的序列输入元素； $\mathbf{W} \in \mathbb{R}^{J \times I}$ 表示输入层的线性变换，一方面对齐输入和隐层的维度，另一方面一次全连接操作便于提取数据特征； $\mathbf{U} \in \mathbb{R}^{J \times J}$ 表示隐层的记忆内容转换，以上两个矩阵变换把输入数据和隐层向量映射到了统一的状态空间中，最后通过激活函数提取出模型当前状态的序列信息，即向量 $\mathbf{h}_t \in \mathbb{R}^J$ 。公式 (2-2) 表示输出层， $\mathbf{o}_t$ 表示序列当前的输出向量。根据应用的不同，模型结构也

① $\sigma(x) = \frac{1}{1+e^{-x}}$

有各种变化，如图2-2所示。

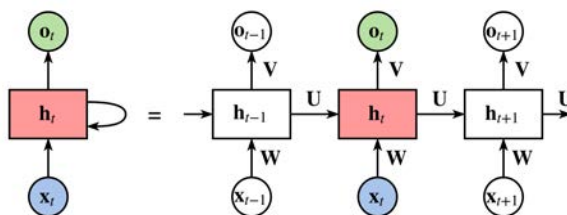


图 2-1 RNN 及其展开形式

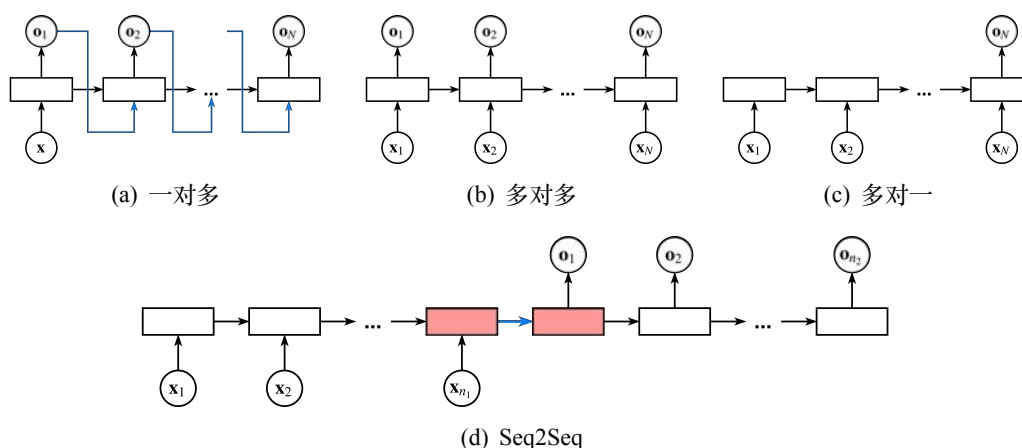


图 2-2 RNN 的几种应用类型。其中 (a) 一对多常用在序列生成领域，如给定第一个音符后生成一段旋律；(b) 多对多常用在序列变换中，如 NLP 领域的词性标注；(c) 多对一常用在序列分类任务中，如视频/音频分类；(d) Seq2Seq 是一个典型的编码器-解码器结构，常用在机器翻译等任务中。

RNN 网络需要使用 BPTT (Back-Propagation Through Time) 算法^[83,84] 进行训练。以一对多网络为例，forward 过程按时序从小到大顺序计算，最终得到 loss 值；然后按照时序从大到小反序计算每一个时刻的梯度，并利用 SGD 等优化方法更新模型参数。而由于模型仅有一份 $\mathbf{W}$, $\mathbf{U}$, $\mathbf{h}$ 参数，因此这三者在每个时刻的 backward 计算中都被反复更新。

2.1.2 LSTM

在实际应用中，由于 $\mathbf{h}_t$ 的梯度 $\frac{\partial \ell}{\partial \mathbf{h}_t} = (\frac{\partial \mathbf{h}_{t+1}}{\partial \sigma_h} \cdot \mathbf{U})^T \cdot \frac{\partial \ell}{\partial \mathbf{h}_{t+1}}$ ，由于梯度中存在 $\mathbf{U}$ 项，在反向传播时出现了连续矩阵乘法，最终导致梯度震荡，出现梯度爆炸或梯度消散问题。在序列越长的模型中，梯度震荡现象越严重，导致模型收敛缓慢，长期记忆力羸弱的问题。因此，在 1997 年 Sepp Hochreiter 等人提出了 LSTM 网络^[81]，把其中的乘法关系转变为点乘关系，解除了梯度计算过程中的连续乘法问题；同

时引入隐层状态遗忘机制，解决了超长序列记忆问题。其模型如公式 (2-3)-(2-8) 描述：

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot \mathbf{x}_t + \mathbf{U}_f \cdot \mathbf{h}_{t-1} + \mathbf{b}_f) \quad (2-3)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot \mathbf{x}_t + \mathbf{U}_i \cdot \mathbf{h}_{t-1} + \mathbf{b}_i) \quad (2-4)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot \mathbf{x}_t + \mathbf{U}_o \cdot \mathbf{h}_{t-1} + \mathbf{b}_o) \quad (2-5)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \cdot \mathbf{x}_t + \mathbf{U}_c \cdot \mathbf{h}_{t-1} + \mathbf{b}_c) \quad (2-6)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (2-7)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (2-8)$$

其中，函数 σ 表示激活函数， $\odot$ 表示逐元素乘法（element-wise product）， $\mathbf{W}_*$ 表示输入与隐层状态间的映射权重矩阵， $\mathbf{U}_*$ 表示上一时刻的隐层状态与当前隐层状态的映射权重矩阵。

LSTM 网络模型的图形表示如图2-3所示。其设计的核心思想是引入了单元状态 $\mathbf{c}_t$ （cell state），由于单元状态仅有较少的线性操作，因此信息能在其中留存且不被改变。同时引入了三个门（gate）对单元状态所保存的信息进行遗忘或更新，如遗忘门（forget gate） $\mathbf{f}_t$ 对保存的信息进行选择性丢弃，输入门（input gate） $\mathbf{i}_t$ 对新输入的信息选择性更新到单元状态中。然而由于输入层矩阵乘法的存在，LSTM 网络^①在面临高维输入数据，即 $\mathbf{x}_t$ 维度较高时， $\mathbf{W}$ 矩阵尺寸激增，带来严重的模型尺寸问题，也使得模型训练和收敛变得困难，本文将在第三章介绍如何解决该问题。

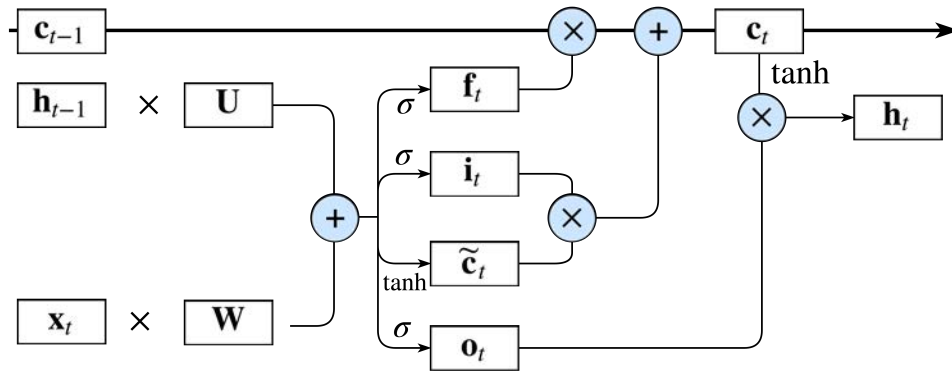


图 2-3 LSTM 网络的图形表示。为了简化图形，把所有门的 $\mathbf{W}_*$ 和 $\mathbf{U}_*$ 矩阵合并为 $\mathbf{W}$ 和 $\mathbf{U}$ 。

① RNN 网络基本都存在类似的问题，这里以 LSTM 为例。

2.2 张量基础

在不同领域上，张量有其特定的含义。在本文中，张量特指多维数组，是向量和矩阵的高维扩展或其广义定义。如图2-4所示，一个张量 $\mathfrak{X} \in \mathbb{R}^{I \times J \times K}$ 拥有 $I \times J \times K$ 个元素。张量在表示高维线性关系，高维数据时，相比矩阵而言有更优秀的表达能力。

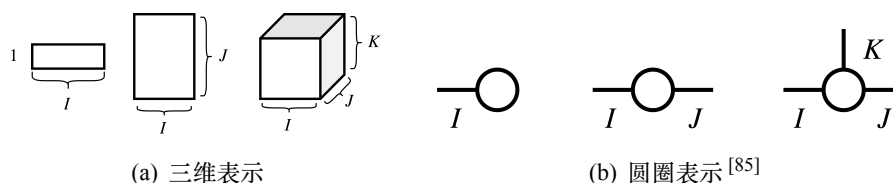


图 2-4 张量的两种图形表示方法。其中圆圈表示法中圆圈边的数量表示了张量的阶数。

2.2.1 张量乘法

正如矩阵和向量有其特定的乘法一样，张量也有其特定定义的张量乘法。通过类比矩阵乘法 $C_{i,j} = \sum_k A_{i,k} B_{k,j}$ ，我们可以方便得出张量乘法的定义。给定张量 $\mathfrak{X} \in \mathbb{R}^{I \times J \times K}$ 和 $\mathfrak{Y} \in \mathbb{R}^{I \times J \times P}$ ，两个张量可以沿着相同维度的边进行现成，如沿着维度 I 和维度 J 的张量乘法定义为：

$$\begin{aligned} \mathfrak{Z}_{k,p} &= \mathfrak{X} \times_{I,J} \mathfrak{Y} \\ &= \sum_{i=1, j=1}^{i=I, j=J} \mathfrak{X}_{i,j,k} \mathfrak{Y}_{i,j,p} \end{aligned} \quad (2-9)$$

其中， $\times_{I,J}$ 表示同时沿两个维度进行乘法。图2-5。 $\mathfrak{X}_{i,j,k}$ 表示其中一个元素。张量乘法还需考虑维度对齐，转置等问题，但并非本文关键内容，因此略去不提，具体内容可参考 [61] 和 [86]。

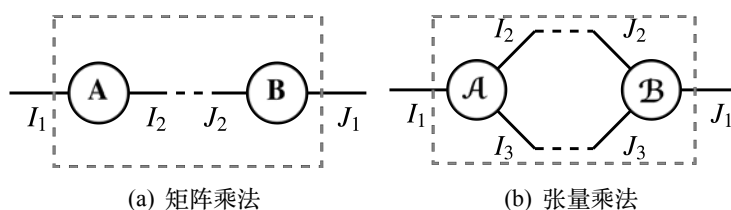


图 2-5 矩阵乘法和张量乘法的图形表示。图中的虚线表示两者维度相同，沿该维做乘法运算。

2.2.2 张量分解

张量分解是矩阵分解的高维扩展。对于一个高维张量 $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_d}$ ，可以将其分解为若干低维张量的乘积关系。相比矩阵分解而言，张量分解能用更少的模型参数达到同样的表达能力。因此本文将尝试使用张量分解技术来完成降维压缩模型的目的。下面介绍两个本文将会用到的张量分解算法。

2.2.3 CP 分解算法

CP 分解算法 (CANDECOMP/PARAFAC, CP) 来源于 1970 年 Carroll 等人提出的 CANDECOMP (canonical decomposition) [87] 和 Harshman 提出的 PARAFAC (parallel factors) [88]，这里根据综述文章 [61] 的命名方式来使用。CP 分解的核心理想是把一个高阶张量分解为一系列秩为 1 (即可被一系列向量的外积所表征) 的张量和，其中张量的个数为 CP 分解中设置的秩，这里我们将它命名为 CP 秩。由于张量的秩的求解是一个 NP-hard 难题，目前尚未有成熟的计算方式，而在实践中通常不需要非常精确地给定，因此在求解 CP 分解时通常 CP 秩作为一个超参给定。给定一个 N 阶张量 $\mathcal{X} \in \mathbb{R}^{I_1 \times \dots \times I_N}$ ，它的 CP 分解为：

$$\mathcal{X} \approx \sum_{r=1}^R \mathbf{A}_r^{(1)} \circ \mathbf{A}_r^{(2)} \circ \dots \circ \mathbf{A}_r^{(N)} \quad (2-10)$$

$$\mathcal{X}_{i_1, i_2, \dots, i_N} \approx \sum_{r=1}^R \mathbf{A}_{i_1, r}^{(1)} \times \mathbf{A}_{i_2, r}^{(2)} \times \dots \times \mathbf{A}_{i_N, r}^{(N)} \quad (2-11)$$

$$\mathcal{X} = [\Lambda; \mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \dots, \mathbf{A}^{(N)}] \quad (2-12)$$

其中， $\mathbf{A}^{(k)} \in \mathbb{R}^{I_k \times R}$, $k \in [1, N]$ ，被称为 CP 分解中的因子矩阵； $\mathbf{A}_r^{(k)}$ 表示矩阵中的第 r 列向量， $\circ$ 表示外积操作，N 个向量的外积是一个 N 阶张量。在某些情况下，还可以加入一个 $\lambda \in \mathbb{R}^R$ 向量作为每一个张量的权重，用来调整每一个因子张量的配重。这里为了简化，默认了这个向量为全一向量。在一些文献中也有使用如公式 (2-12) 的简化方法 [89]，其中 Λ 是全一向量。图2-6是一个三阶情况下的 CP 分解示意。

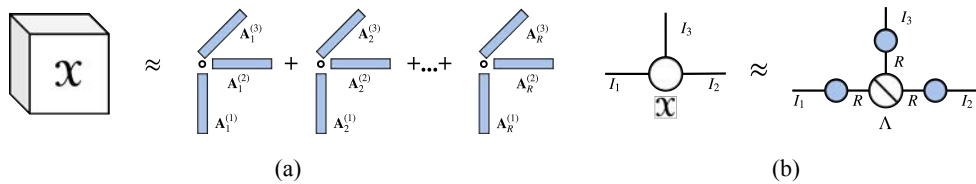


图 2-6 三阶 CP 分解的两种图形表示形式。其中 $\Lambda \in \mathbb{R}^R$ 向量为全一向量，圆圈中有一斜线表示这是一个对角张量，只有体对角线上有值，其余为 0。

求解 CP 分解是一个非凸优化问题，一般使用交替最小二乘法（Alternating Least Squares, ALS）进行求解，其核心思想是：在面临多参数求解时，每一步选择某一个参数进行求解，其余参数固定，此时退化为一个单一变量的求解问题，容易求得最优值；下一步交替选择其他参数进行求解。交替最小二乘法求解简单，收敛速度较快，然而由于存在自由度过高的问题，模型的收敛质量一般。近年来随着深度学习的发展，尤其是非凸优化的研究进步，现在通常使用基于梯度下降的求解算法，如 Adam^[90]、SGDm^[91] 等。此外，CP 分解算法由于存在 CP 秩求解困难，直接给定 CP 秩进行求解又容易导致病态的 Forbenius 范数出现^[92]，因此逐渐被新一代张量分解技术，如 Tensor Train 分解^[64]、Tensor Ring 分解^[93] 等替代。

2.2.4 Tucker 分解算法

Tucker 分解是矩阵领域的奇异值分解（Singular Value Decomposition, SVD）的高阶扩展，在 1963 年被 Tucker 首次提出^[94]，是一种稳定的分解算法。在一些文献中，Tucker 分解亦被称为 Higher-order SVD（HOSVD）^[95] 或高阶 PCA（Principal Components Analysis）^[61]。给定一个 N 阶张量 $\mathcal{X} \in \mathbb{R}^{I_1 \times \cdots \times I_N}$ ，它的 Tucker 分解为：

$$\mathcal{X} \approx \mathcal{G} \times_1 \mathbf{A}^{(1)} \times_2 \mathbf{A}^{(2)} \times_3 \cdots \times_N \mathbf{A}^{(N)} \quad (2-13)$$

$$\mathcal{X}_{i_1, i_2, \dots, i_N} \approx \sum_{r_1, r_2, \dots, r_N=1}^{R_1, R_2, \dots, R_N} \mathcal{G}_{r_1, r_2, \dots, r_N} \mathbf{A}_{i_1, r_1}^{(1)} \mathbf{A}_{i_2, r_2}^{(2)} \cdots \mathbf{A}_{i_N, r_N}^{(N)} \quad (2-14)$$

$$\mathcal{X} = [\![\mathcal{G}; \mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \dots, \mathbf{A}^{(N)}]\!] \quad (2-15)$$

其中， $\mathcal{G} \in \mathbb{R}^{R_1 \times R_2 \times \cdots \times R_N}$ 表示低维的核心张量，可类比为 SVD 的奇异值对角矩阵； $\mathbf{A}^{(k)} \in \mathbb{R}^{R_k \times I_k}, k \in [1, N]$ 表示第 k 个因子矩阵，通常要求是正交矩阵，可类比为 SVD 的主成分； R_k 表示该分解中的秩，通常远小于 I_k 以达到降维压缩的目的。一些文献会使用公式 (2-15) 的简写来表示 Tucker 分解^[89]。图 2-7 是 Tucker 分解的两种图形表示形式。当核心张量 $\mathcal{G}$ 设置为对角张量，且 $R_1 = R_2 = \cdots = R_N$ ，则 Tucker 分解退化为 CP 分解。

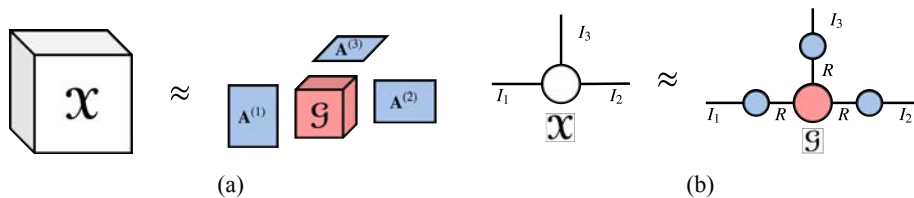


图 2-7 三阶 Tucker 分解的两种图形表示形式

求解 Tucker 分解的方法一般也是根据 SVD 的求解方式推广开来，如 HOSVD

算法，它需要先计算每一维奇异值向量，然后才计算核心张量；另外一种方法是 Higher-order Orthogonal Iteration (HOOI)，其核心思想是利用了交替最小二乘法求解，而过程公式可见文献 [61]，这里不再赘述。Tucker 分解存在一个较为严重的问题是 Tucker 分解是不唯一的，考虑一个三阶 Tucker 分解的例子： $[[\mathcal{G}; \mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \mathbf{A}^{(3)}]] = [[\mathcal{G} \times_1 \mathbf{U} \times_2 \mathbf{V} \times_3 \mathbf{W}; \mathbf{A}^{(1)}\mathbf{U}^{-1}, \mathbf{A}^{(2)}\mathbf{V}^{-1}, \mathbf{A}^{(3)}\mathbf{W}^{-1}]]$ ，如图2-8所示。不唯一性导致了 Tucker 分解存在性能不稳定的问题，一方面在应用中通常需要进行额外的约束，另一方面也带来了简化分解的可行性，如添加约束使核心张量 $\mathcal{G}$ 稀疏化，降低训练难度

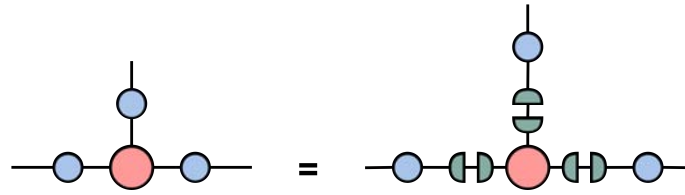


图 2-8 Tucker 分解不唯一示意图，图中两个半圆表示一对互逆矩阵，如 $\mathbf{U}$ 和 $\mathbf{U}^{-1}$ 。

2.2.5 Tensor-Train 链式分解

无论是 CP 分解还是 Tucker 分解，都涉及到秩设置的问题，而通常高阶张量的秩求解非常困难，难以达到准确分解的目的。在 2011 年 Oseledets 等人提出了 Tensor-Train [64]，一种结构简单，计算高效的分解算法，其分解思想来源于量子领域的 MPS (Matrix Product State) [96]。作者认为，在面临超高维数据，如上百维、上千维的张量时（这通常发生在量子模拟、随机偏微分方程求解、金融领域建模等应用中），由 Tucker 分解获得的核心张量秩将以次方级别增长（核心张量的参数数量是 R^d ），导致现阶段任何计算机都无法模拟和计算。因此借鉴了 MPS 公式的思想，构建了一种结构均匀的分解方式，即 Tensor-Train 分解。它的 d 阶分解公式如下：

$$\mathcal{X} \approx \mathcal{G}^{(1)} \times_2^1 \mathcal{G}^{(2)} \times_2^1 \cdots \times_2^1 \mathcal{G}^{(d)} \quad (2-16)$$

$$\mathcal{X}_{i_1, i_2, \dots, i_d} \approx \sum_{r_0, r_1, r_2, \dots, r_d} \mathcal{G}_{r_0, i_1, r_1}^{(1)} \mathcal{G}_{r_1, i_2, r_2}^{(2)} \cdots \mathcal{G}_{r_{d-1}, i_d, r_d}^{(d)} \quad (2-17)$$

其中， $\mathcal{G}^{(k)} \in \mathbb{R}^{R_{k-1} \times I_k \times R_k}, k \in [1, d]$ ，表示其中一个核心张量；其中 $R_0 = R_d = 1$ 。其图形结构如图2-9所示。可以发现，它有多核心张量，每一个都是一个三阶张量，其中两端张量有一个为 1 的维度，即 R_0 和 R_d ，退化为矩阵形式。

Tensor-Train 的形式简洁，可以使用矩阵的奇异值分解等快速算法进行求解，近年来受到了广泛的关注 [62, 65, 66]。然而从“信息瓶颈” [67, 97] 的角度看^①，Tensor-

① 简言之，考虑矩阵乘法 $C = AB$ ，则 $\text{rank}(C) \leq \min(\text{rank}(A), \text{rank}(B))$ ，即相乘会导致秩下降。

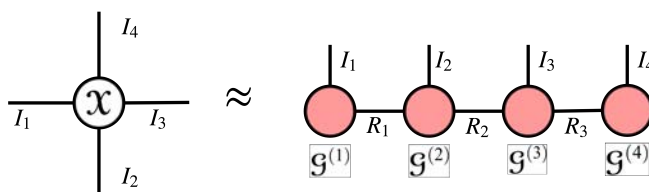


图 2-9 四阶 Tensor-Train 分解图形示意

Train 的秩 $R_0 \dots R_d$ 要求是两端小，中间大，导致了超参数过多的问题。而在实践中，通常忽略了这点，可能带来精度上的损失。本文将在第三章中详细介绍，并在实验中证明精度损失的问题。

2.2.6 Block-Term 块张量分解

块张量分解算法 (Block-Term Tensor Decomposition, BT 分解) [69] 是 CP 分解和 Tucker 的结合，解决了 CP 分解的转置不稳定，以及 Tucker 分解表达能力不足的问题，同时能抑制输入数据的噪音影响 [68]。其模型结构是若干个 Tucker 分解模型的和：

$$\mathcal{X} \approx \sum_{n=1}^N \mathcal{G}_n \times_1 \mathbf{A}_n^{(1)} \times_2 \mathbf{A}_n^{(2)} \times_3 \dots \times_d \mathbf{A}_n^{(d)} \quad (2-18)$$

$$\mathcal{X}_{i_1, i_2, \dots, i_d} \approx \sum_{n=1}^N \sum_{r_1, r_2, \dots, r_d}^{R_1, R_2, \dots, R_d} \mathcal{G}_{n, r_1, r_2, \dots, r_d} \mathbf{A}_{n, i_1, r_1}^{(1)} \mathbf{A}_{n, i_2, r_2}^{(2)} \dots \mathbf{A}_{n, i_d, r_d}^{(d)} \quad (2-19)$$

其中， $\mathcal{G}_n \in \mathbb{R}^{R_1, R_2, \dots, R_d}$ 表示第 n 个 Tucker 分解模型的核心张量， $\mathbf{A}_n^{(k)}, k \in [1, d]$ 表示第 n 个 Tucker 分解模型的第 k 个因子矩阵； N 表示 Tucker 分解模型的个数，在本文中我们称之为 BT 分解的 CP 秩； $R_k, k \in [1, d]$ 表示 BT 分解的 Tucker 秩，在一些使用场景下通常可以设置 $R_1 = R_2 = \dots = R_d$ ，从而避免了不同维度间的权重配比不一致问题； d 表示 BT 分解的阶。当 $N = 1$ 时退化为标准的 Tucker 分解，当 $N = 1$ 且核心张量为对角张量时，退化为 CP 分解。其图形表示如图2-10所示，BT 分解算法将一个张量分解为 N 个块；每一个块是一个完整的 Tucker 分解模型。在某些场合下为了配比不同块的权重，会加入一个 $\lambda \in \mathbb{R}^N$ 向量调整线性加和的权重。本文并未用到此特性，因此略去。本文将在第三章详细阐述 BT 分解相比于 Tensor-Train 分解的一些优势和不足，以及相应的实验对比。

2.3 卷积神经网络

卷积神经网络 (Convolutional Neural Networks, CNN) 最初的模型结构由 Kunihiro Fukushima 等人于 1980 年提出 [10]，而其核心的“视觉感知域” (Receptive

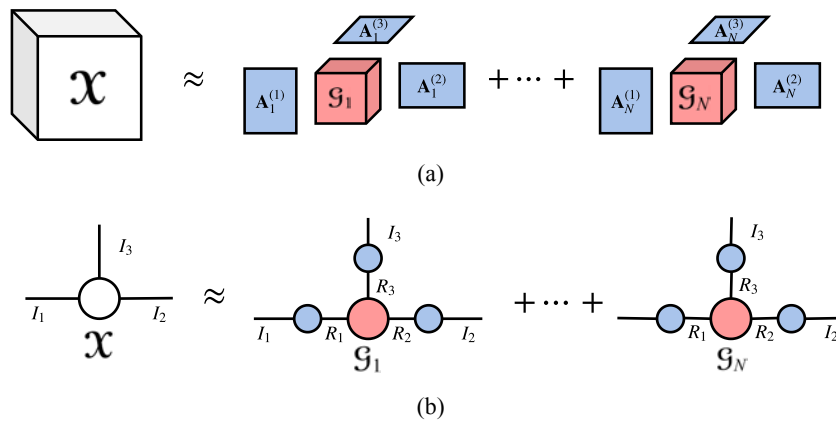


图 2-10 BT 分解的两种图形表示

Fields) 最初由 Hubel 和 Wiesel 于 1968 年提出^[30], 其思想是充分利用图像的空间局部性, 每个神经元没有必要对全局像素进行感知, 只需对一个小区域内的像素进行感知, 然后在高层视野中讲底层视野感知的信息组合, 即可得到高层的视觉感知信息。由此引入了两个关键操作, 卷积操作 (Convolution) 和池化操作 (Pooling), 卷积操作用于感知部分图像区域, 提取视觉特特征信息; 而池化操作则是将局部区域信息聚合, 降低数据处理维度。CNN 网络则是由卷积层和池化层交替构建而成, 中间还需要引入非线性的激活函数如 ReLU^[98]、sigmoid 等, 从而形成了深度神经网络的概念。卷积和池化操作的过程如图2-11所示, 而一个完整的 CNN 网络请见图2-12。

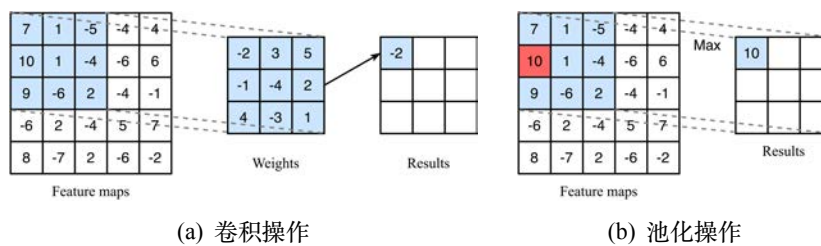


图 2-11 卷积神经网络的两个关键操作

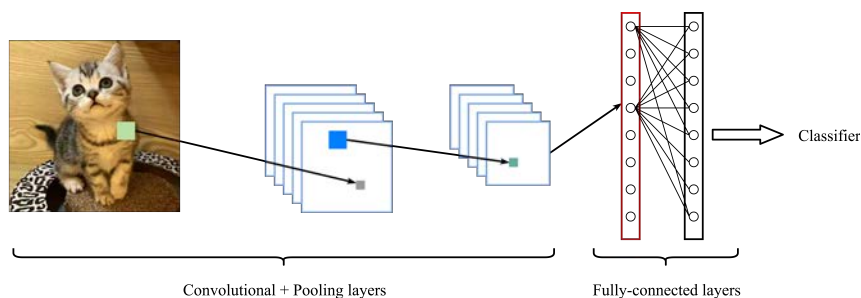


图 2-12 卷积神经网络

近年来研究发现，传统的线性结构 CNN 网络存在梯度传递困难，训练效率低下的问题^[34]，因此引入了各种非线性结构网络模型，图2-13列出了两种较为常见的非线性结构模型，其中 (a) 是扇形结构，最初由 Inception 网络^[99] 提出，把通道拆分后处理，一方面有利于提高通道利用率，另一方面可以降低网络深度，加宽了网络宽度，然而这种结构极大增加了模型的计算密度，已经被逐渐弃用；(b) 是联合结构，首次在文章 Highway Networks^[100] 中提出，通过加入短路连接结构，可以解决深度网络梯度反传困难的问题，并被 ResNet^[37] 借鉴使用，成为新一代网络的设计规范；后来 DenseNet^[101] 扩展了这种思想，将每一层的感知结果全部上传到网络的高层，在高层实现特征重用，极大提升了图像的特征利用率，从而减少了模型参数，有利于降低模型复杂度。

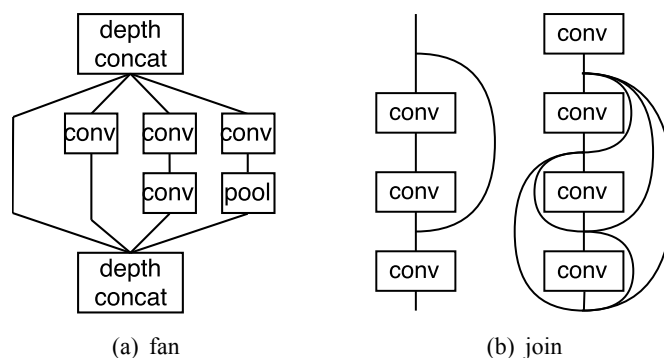


图 2-13 两种较为常见的非线性结构模型，可见每一层的前后续不再只有一个层，变量依赖将变得极其复杂。

2.3.1 CNN 模型训练过程

一个 CNN 网络由若干不同类型的层前后接续构成，在数学表达上是一个复杂的复合函数，因此复合函数的求导规则和链式法则同样适用于 CNN 网络，以此为基础构建的误差反向传播算法（Backpropagation, BP）最早于 1986 年由 Rumelhart 和 Hinton 等人引入到神经网络领域中^[12]。BP 算法分为两步，第一步是 forward 前向传播，计算模型误差项；第二步是根据误差项 backward 反向传播，计算每一项参数的梯度值，并更新模型参数。图2-14描述了一个 CNN 网络的前向-反向传播过程。图中的 softmax 层用于计算误差项，误差将通过红色虚线箭头方向逐步计算每一层的参数梯度并更新。

由于 CNN 网络是一个高度非线性非凸函数，难以获得全局最优解。因此通常使用梯度下降算法，以较小的学习率（learning rate）进行迭代更新。同时由于训练样本通常含有较多噪音，且为了降低模型训练难度，一般采用随机梯度下降算法

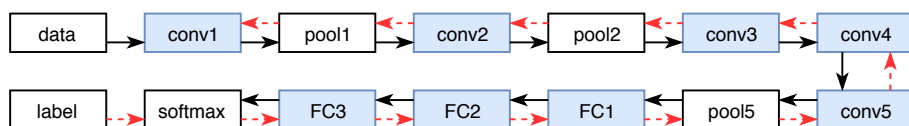


图 2-14 前向-反向传播算法过程。黑色实线箭头表示前向算法的计算顺序，红色虚线箭头表示反向算法的计算顺序。蓝色的层表示含有模型参数，在反向计算时需要同时更新模型参数。

(Stochastic Gradient Descent, SGD) 进行训练。以 SGD 算法为基础，研究人员提出了一系列改进算法，如 Adam^[90]，SGDm（带 momentum 项的 SGD 算法）^[91]，NAG（Nesterov accelerated gradient）算法^[102]等，以解决算法在面临鞍点时的不稳定性问题，同时加快训练速度^[103]。下面仅以基础的 SGD 算法为例说明 CNN 模型的训练过程。

给定一个模型 $f(x, \omega)$ ，模型的输入数据是 x ，对应的标签是 y ；模型参数是 ω ，学习率为 η 。SGD 算法每次执行将从训练数据集中随机采样 b 个样本组成小批量数据，forward 过程中根据模型构建时的先后层次关系，计算得到损失函数值 ℓ ；在 backward 过程中将根据求导的链式法则反向计算并更新每一层参数的梯度。更新的过程为：

$$\omega := \omega - \eta \frac{1}{b} \sum_{i=1}^b \nabla f(X_i, \omega) \quad (2-20)$$

其中， X_i 是一次小批量更新中的第 i 个训练样本， $\nabla f(X_i, \omega)$ 是函数 $f(X_i, \omega)$ 关于 ω 的梯度。

2.3.2 设备内存

为了加快训练速度，目前通常使用计算卡加速计算，如 GPU (Graphics Processing Unit)。目前 NVIDIA GPU 被广泛使用，在这上面设计的 CUDA 平台^①和 CUDNN 运行库^[80]成为各大主流训练框架的基础设施之一。本文将以 NVIDIA GPU 为例，设计并实现我们的动态内存管理方案，相关技术同样适用于类似的计算平台。

如图2-15所示，我们将设备分为了 CPU 端和 GPU 端^②，中间将通过 PCI-e 通道^③相连。CPU 端计算核比较少，但功能强大，适用于各种复杂任务，且设备内存通常较大，通常具有上百 GB；而 GPU 端计算核数量众多，如最新的 NVIDIA

① <https://developer.nvidia.com/cuda-toolkit>

② 其他通过 PCI-e 通道连接的计算卡设备同样适用此模型。

③ <http://pcisig.com/>

TITAN V 卡拥有 5120 个核，适用于简单重复的数学计算，但是设备内存很小，只有 12GB^①。而训练神经网络通常需要耗费几十甚至上百 GB 内存，因此需要频繁调度 CPU-GPU 端的内存。而由于 PCI-e 通道传输速度远低于 CPU 和 GPU 的计算速度，因此如何合理调度内存传输，从而降低设备等待时间，加快训练速度，成为了各大主流计算框架的核心设计任务。本文也是基于此原因设计并实现了一套动态内存管理方案，以期望降低 GPU 端内存压力，提升内存使用效率，降低等待时间，加快训练速度。

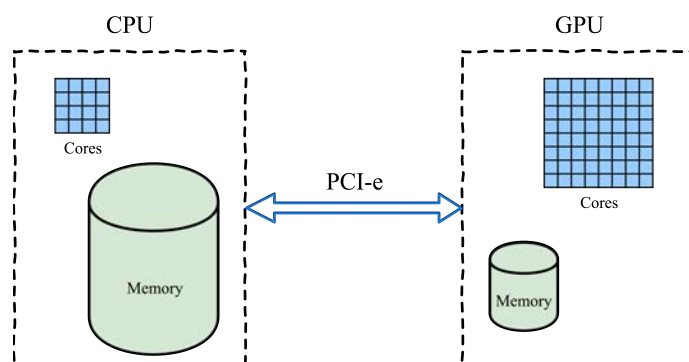


图 2-15 CPU-GPU 结构。其中 CPU 端拥有较少的计算核，但是有非常大的内存空间；GPU 端有众多的计算核，但是设备内存空间很小。两端通过 PCI-e 通道相连。

2.3.3 模型的内存观点

神经网络被设计成层级结构，我们也将以层为单位，从内存观点方面解释计算框架在内存调度方面存在的苦难和现有解决方案。这里为了分析方便，我们将以全连接层（Fully-connected, FC）， $y = Wx$ 为例说明^②。假设当前 FC 层是模型的第 l 层，在该层的 forward 和 backward 过程中，将涉及到以下变量：

- 当前层的输入 x^l ，也是前一层的输出变量；
- 当前层的模型参数 ω^l ，在 FC 层中是 W ；
- 当前层的模型梯度 $\nabla\omega^l$ ，在 FC 层中是 ∇W ；
- 传回到上一层的误差项 ∂f^l ，将被上一层用于计算梯度及误差反传；
- 当前层的输出 x^{l+1} ，也是下一层的输入变量；
- 下一层传回的误差项 ∂f^{l+1} ，将被当前层用于计算 $\nabla\omega^l$ 和 ∂f^l 。

其过程如图2-16所示。在 forward 过程中，FC 层根据输入 x^l 和模型参数 W 计算得到输出值 y^l ，输出值将传递到下一层用于计算；在 backward 过程中，根据下一层

① <https://www.nvidia.com/en-us/titan/titan-v/>

② Conv 层同样适用，但其他类型的层可能不适用于此分析，如 Pooling 层没有模型参数，输入层无需梯度反传，这里为了简化分析而以 FC 层为例。

传回的误差项 ∂f^{l+1} 计算模型梯度 $\nabla \omega^l$ 和传回上一层的误差项 ∂f^l ，并更新参数 W 。我们发现，在 forward 计算过程的中间变量 x^l 和 x^{l+1} 仍被 backward 所需要，因此单单 FC 层在计算过程中，backward 时会达到内存峰值，需要满足除 x^{l+1} 以外所有变量的内存需要。

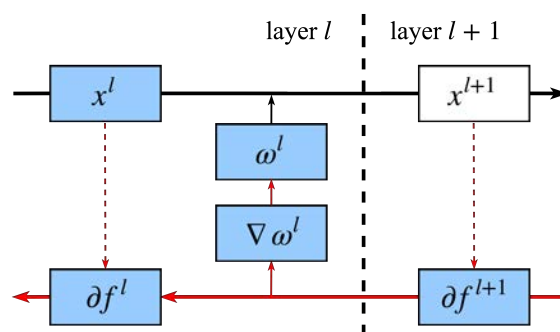


图 2-16 全连接层变量利用过程。图中黑色箭头表示前向算法计算过程，红色箭头表示反向算法计算过程。红色虚线表示反向计算过程中需要依赖的变量。蓝色变量表示在 backward 过程中当前层的内存峰值情况。

通过上述分析，不难发现一个层占据的内存远比想象的要大，在 2012 年发表的 AlexNet^[33]，就是因为单卡内存不足，需要把模型拆分成两个子模型，分布在两张计算卡上执行。而新一代的 CNN 网络，如 ResNet152 和 InceptionV4，在 batch-size 为 32 时，分别需要消耗 18.5GB 和 44.3GB 内存^[28]，远超 GPU 上的设备内存大小。因此需要针对神经网络训练特性来设计新的内存管理技术，以满足越来越庞大的内存需求。

2.3.4 主流平台的内存管理方案

这里我们分析主流深度学习训练平台，如老一代的 Caffe^[74] 和 Torch^[73]，以及新一代的 TensorFlow^[72] 和 MxNet^[75]，它们各自在内存管理方面所涉及的技术。其中老一代的平台是以层为单位进行内存管理和计算，新一代的平台基本都是以图计算引擎为底层实现技术，以 Operator 为单位进行内存管理和计算。为了简化行文，本文将以层为单位进行叙述，相关技术同样适用于基于图计算引擎的平台^①。

内存重用技术：根据前文分析，backward 过程中需要依赖 forward 的计算结果，而在当前层 backward 计算技术后 forward 的结果将被弃置，因此可以把这部分弃置的内存重用，如使用 x^l 的内存存储 ∂f^l 。这项技术在线性结构的网络中非常实用，理论上可节省高达 50% 的内存，极大地缓解了内存压力，Caffe 和 Torch

① 把 Operator 当做层来考虑，相关技术同样适用

都有相关实现^①。然而在面对非线性结构的网络模型时，层级结构不再是简单的前后顺序，变量的依赖关系变得相当复杂，不能简单地复用同层变量内存，需要仔细控制每一个变量的生存期，相关技术我们将在第四章详细讨论。

基于有向图的变量生存期分析技术：图计算引擎将模型描述为一个有向无环图（Directed Acyclic Graph, DAG），粒度较大的层级结构被拆分成粒度更细的 Operator 和 Tensor 结构，因此对每一个变量而言都有足够丰富的前后顺序依赖关系，便于引擎管理。当发现某些变量不再需要时，将能立刻释放，及时回收内存，已在 TensorFlow 和 MxNet 中被实现和应用。

再计算策略：在 MxNet 中实现了一个类似 Spark 的 RDD（Resilient Distributed Datasets）结构的在计算策略（re-computation），其核心思想是把网络中计算耗时较短的“廉价”层在 forward 计算完后立刻释放变量，在 backward 计算时再重新执行一遍 forward 获得所需变量，其过程如图2-17所示。虽然再计算策略能在一定程度上降低设备内存使用，然而却是以牺牲了训练速度为前提的，因此其实用性值得深思。

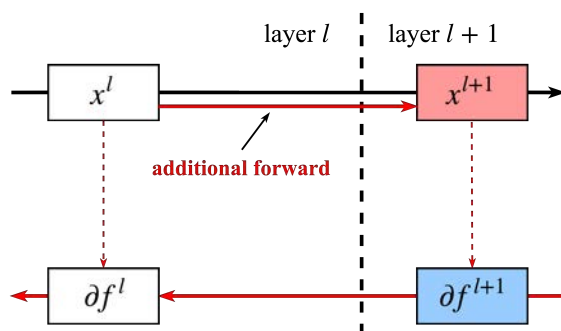


图 2-17 再计算策略。在计算 ∇f^{l+1} 时需要依赖 x^{l+1} ，然而该变量在 forward 过程中被释放，因此需要额外一次 forward 过程计算获得 x^{l+1} 。

2.4 本章小结

本章着重介绍了一些与本文相关的背景知识，这些背景知识将在第三章和第四章被应用。如第三章叙述的张量循环神经网络，就需要依赖循环神经网络和张量基础方面的背景知识。第四章动态内存管理技术方面需要了解网络训练的完整过程，以及与其息息相关的内存观点。

循环神经网络适用于处理序列信息，但是其模型结构相当简单，而输入层则是一个简单的全连接层（矩阵乘法），当面临高维数据输入，如端到端训练的视频分类任务中就需要面对极高维的输入，导致了输入层的参数量过大，受限于硬

^① http://mxnet.incubator.apache.org/versions/master/architecture/note_memory.html

件设备的内存，模型训练将变得极其困难。为此我们引入了张量分解方法，将庞大的输入层重整化为一个高维张量，再应用张量分解技术，分解为若干小型张量，极大程度上降低了模型参数量，缓解了设备内存压力，使得更为庞大的模型训练成为可能。相关方法将在第三章详细介绍。

在设备资源受限的情况下完成模型训练任务，改变模型结构是一个相对风险较高的方法，同时会带来模型精度问题。另一个方法则是通过合理的内存调度方案，充分利用计算平台的所有资源，尽可能加速模型训练过程。现阶段主流的深度学习训练平台，如 TensorFlow、MxNet 等都为此做了不少的努力，我们将另辟蹊径，针对非线性结构网络的特殊性，设计一套新的内存调度管理方案，以期在同等硬件设备情况下训练更大、更深的网络模型，同时维持合理的训练速度。相关技术将在第四章详细介绍。

第三章 张量循环神经网络

循环神经网络（RNN）的模型构造简单，其核心思想是利用一个隐层状态向量保存当前的序列信息，并利用单层非线性变换捕捉当前序列输入数据的序列特征信息。然而当 RNN 在处理高维数据输入时会出现严重的模型膨胀问题。如在端到端训练的视频分类任务中，输入序列是视频在时序上的图像帧。以 UCF11 视频分类数据集^[104]为例，每一帧图像大小是 $320 \times 240 \times 3 = 230,400$ 。假设隐层状态大小为 256，若使用端到端训练方式，则输入层的权重矩阵 $\mathbf{W}_*$ 大小为 $230,400 \times 256 \times 4 \approx 235M$ ^①。庞大的模型参数量将导致一方面模型尺寸过大，带来了极高的计算复杂度和内存开销，且容易超过硬件资源的承载极限；另一方面过高的参数量使得传统的优化方法效率极其低下，使得模型训练收敛变得异常困难。

本章将使用张量乘法替换原矩阵乘法，并使用张量分解方法降低权重矩阵大小，尝试解决 RNN 在面临高维数据输入时的的问题。下面将模型构建过程分为三步描述：1. 张量化参数和输入，把输入向量和参数矩阵转换为高维张量，以便后续应用张量理论进行模型裁剪，并将原矩阵乘法替换为张量乘法描述；2. 对参数张量应用张量分解算法，大幅度裁剪模型参数；3. 使用传统的 BPTT（Backpropagation Through Time）^[83,84] 训练方法训练模型。

3.1 张量化操作

原 RNN 模型中，输入是向量 $\mathbf{x}$ ，输入层的参数矩阵是 $\mathbf{W}$ ，为了应用张量分解技术，首先需要把它们转化为张量。张量化操作（Tensorization），也即把向量、矩阵等转换为高维张量的过程。一般有两种方法，一种是“重整形”（reshape），不会改变参数个数；另一种方法是“堆叠”（stack），即把向量或矩阵按照某一维度方向进行堆叠，会增大参数个数。而本章方法是针对资源受限情况下的模型研究，因此将选用不会改变参数个数的“重整形”方法。其示意如图3-1和图3-2所示。

对于 RNN 网络而言，给定输入层参数矩阵为 $\mathbf{W} \in \mathbb{R}^{I \times J}$ ，可以将之张量化为一个 $2d$ 维张量 $\mathcal{W} \in \mathbb{R}^{I_1 \times J_1 \times I_2 \times \dots \times I_d \times J_d}$ ，其中 $I = I_1 \times \dots \times I_d$ ， $J = J_1 \times \dots \times J_d$ 。而对应的 t 时刻输入向量亦可以转换为 $\mathcal{X}_t \in \mathbb{R}^{I_1 \times \dots \times I_d}$ 。那么对于原矩阵乘法，可以转换为对应的张量乘法，见公式 (3-1) 和图3-3。

$$\mathbf{y}_t = \mathcal{W} \times_{1,2,\dots,d} \mathcal{X}_t \quad (3-1)$$

① 本文将以 LSTM 网络为例，GRU 等其他重要变种同样适用本文提出的理论和方法，并不赘述。若使用 4 字节浮点数，同时考虑梯度的内存占用，单单权重矩阵就需要消耗 1.8GB 内存。

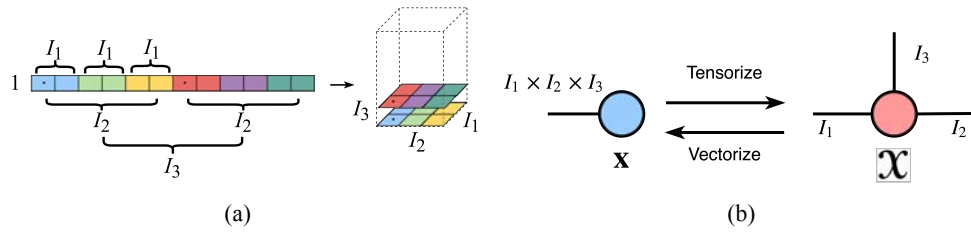


图 3-1 针对向量的三维张量化操作：把一个向量 $\mathbf{x} \in \mathbb{R}^{I_1 \cdot I_2 \cdot I_3}$ 张量化为一个三维张量 $\mathbf{X} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$ 。相反方向的转换则为向量化（Vectorize）。

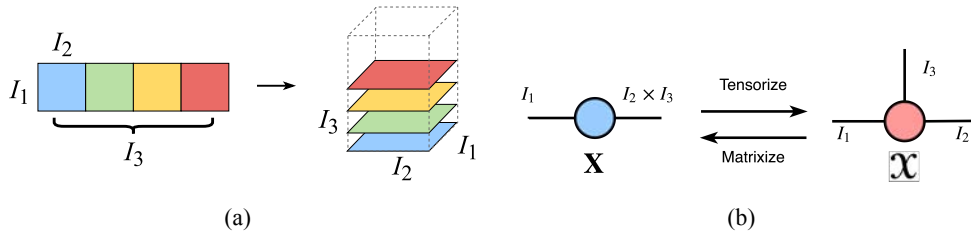


图 3-2 针对矩阵的三维张量化操作：把一个矩阵 $\mathbf{A} \in \mathbb{R}^{I_1 \times (I_2 \cdot I_3)}$ 张量化为一个三维张量 $\mathbf{X} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$ 。相反方向的转换则为矩阵化（Matrixize），在一些文献^[61]中也称为“压平”（Flatten）操作。

其中， $\text{vec}(\mathbf{y}_t) \approx \mathbf{W}^T \mathbf{x}_t$ ， $\text{vec}(\cdot)$ 是向量化操作，见图3-1的“vectorize”操作。

3.2 张量分解算法的应用

虽然原参数矩阵和矩阵乘法被张量方法替换，然而模型的参数量仍未改变，仍然存在模型尺寸过大导致训练困难的问题。然而图像数据通常具有明显的低秩特性和局部结构，导致了参数张量必然具有低秩特性。因此可以对参数张量进行低秩分解，降低模型参数量。下面将引入块张量分解算法（BT 分解算法）对参数张量进行分解，改进原有 RNN 网络结构。据我们所知，这种分解算法是首次被引入到深度学习领域，并获得了超过以往所有同类方法的精度和参数压缩比 [68]。

3.2.1 块张量循环神经网络

块张量分解算法（Block-Term Tensor Decomposition, BTDD）解决了 CP 分解的转置不稳定，以及 Tucker 分解表达能力不足的问题，同时能抑制输入数据的噪声影响。在 RNN 输入层的线性变换中，这些特性尤其重要。例如若分解存在转置不稳定现象，则图片的旋转变换将带来严重的误差；同时图像中普遍带有噪音。而块张量分解算法能避免这些问题，使得模型收敛的精度更好。同时相比 Tensor-Train 分解算法而言，超参数设置上要求更低，以同等参数量的情况下，BT 分解的表达

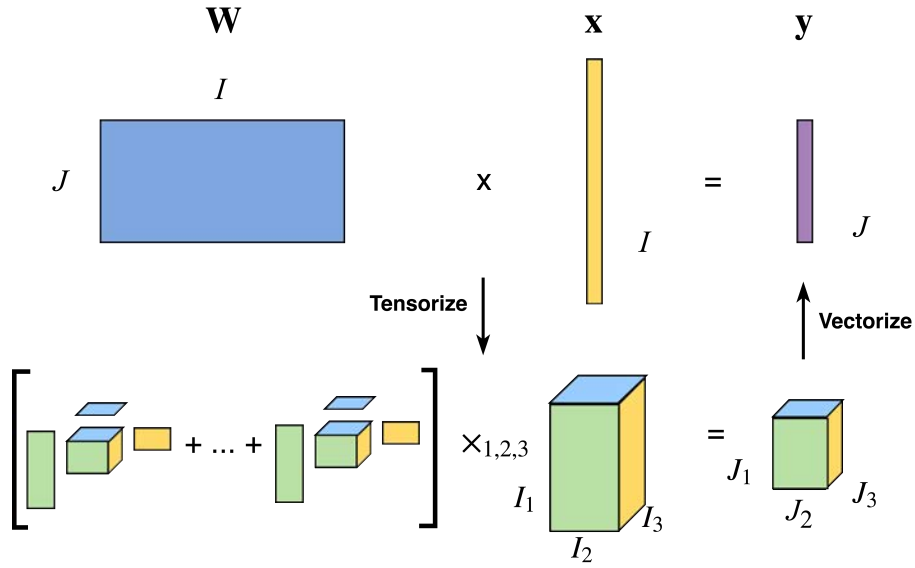


图 3-3 把矩阵乘法转换为张量乘法的过程

能力更强，更适合于神经网络模型。

3.2.1.1 模型构建

对于输入层的张量乘法过程（公式 (3-1)），我们对参数张量 $\mathcal{W}$ 进行块张量分解，即：

$$BTD(\mathcal{W}) = \sum_{n=1}^N \mathcal{G}_n \times_1 \mathcal{A}_n^{(1)} \times_2 \cdots \times_d \mathcal{A}_n^{(d)} \quad (3-2)$$

其中核心张量 $\mathcal{G}_n \in \mathbb{R}^{R_1 \times \cdots \times R_d}$ ，其中 $R_k, k \in [1, d]$ 称为 BTD 分解的 Tucker 秩； $\mathcal{A}_n^{(d)} \in \mathbb{R}^{I_d \times J_d \times R_d}$ 表示因子张量， N 是 BTD 分解的 CP 秩； d 是 BTD 分解的核心阶。为了简化模型超参数设置，同时避免权重分配不均衡问题，我们通常设置 Tucker 秩相等为： $R_k = R, k \in [1, d]$ 。下面如不加说明，则假设 Tucker 秩相等。

$$\mathcal{Y}_t = \sum_{n=1}^N \mathcal{G}_n \times_1 \mathcal{A}_n^{(1)} \times_2 \cdots \times_d \mathcal{A}_n^{(d)} \times_{1,2,\dots,d} \mathcal{X}_t \quad (3-3)$$

此时，原矩阵乘法将被张量乘法替换，见公式 (3-3)。当完成了张量乘法后，我们得到了输出 $\mathcal{Y}_t$ 张量，再完成一次向量化操作，即可获得适用于 RNN 模型公式的向量结果。这样设计的一个好处是尽可能保留了 RNN（包括 LSTM 和 GRU 等变种）其本身的设计原则，适用范围得到保障；同时大幅度降低了模型参数量，降低了模型训练的难度。图3-4是全新的张量化 LSTM 网络，称之为 BT-LSTM。需要注意的是，我们把输入层替换为张量分解的版本后，模型仍然按照端到端的训

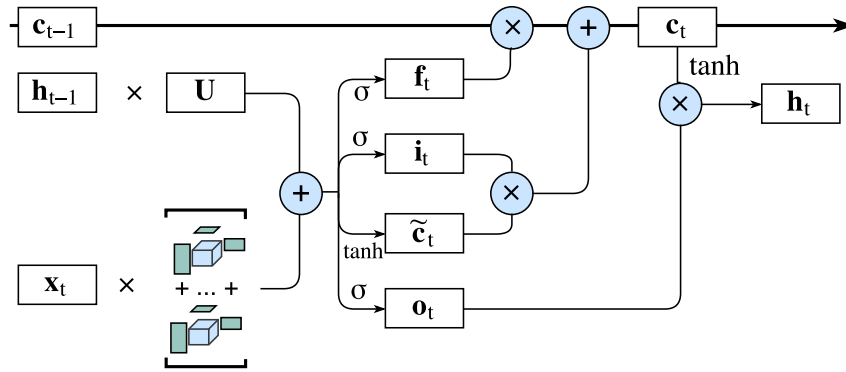


图 3-4 BT-LSTM 模型。注意到新的模型仅改变了输入层的矩阵乘法，其余部分仍然遵循传统的 LSTM 设计原理不变，得以兼容绝大部分 LSTM 适用的领域。

练方式进行训练，相当于我们沿用了 BT 分解的模型结构，弃用了它的求解过程。

3.2.1.2 训练细节

RNN 网络使用 BPTT 算法进行训练，即按照时序递增的方向进行前向传播，并计算损失函数，获得损失值；然后按照时序递减的方向逐一计算每一时刻的梯度值，并更新模型参数。为了简化模型训练过程，复用已有实现，我们将在 BPTT 算法框架下推导出 BT-RNN (Block-Term RNN) 的梯度公式，并在后续“模型复杂度分析”小节上分析其时间和空间复杂度。为了简化计算，我们将只关注输入层的权重参数的梯度求导，其余参数的梯度公式与传统 RNN 一致。为了简化计算，我们将推导按元素进行的梯度公式。公式 (3-4) 是按元素乘法的 BT 分解模型。

$$\mathbf{y}_{\vec{j}} = \sum_{n=1}^N \sum_{\vec{r}}^{R_1, \dots, R_d} \sum_{\vec{i}}^{I_1, \dots, I_d} \prod_{k=1}^d \mathbf{x}_{t, \vec{i}} \mathcal{A}_{n, i_k, j_k, r_k}^{(k)} \mathcal{G}_{n, \vec{r}}. \quad (3-4)$$

这里为了简化公式，我们使用了 $\vec{i}, \vec{j}$ 和 $\vec{r}$ 分别表示下标 $(i_1, \dots, i_d)$, $(j_1, \dots, j_d)$ 和 $(r_1, \dots, r_d)$ 。由于公式两端均是标量，因此其对应的梯度公式如下：

$$\frac{\partial L}{\partial \mathcal{A}_{n, i_k, j_k, r_k}^{(k)}} = \sum_{\vec{r} \neq r_k} \sum_{\vec{i} \neq i_k} \sum_{\vec{j} \neq j_k} \prod_{k' \neq k}^d \mathcal{G}_{n, \vec{r}} \mathcal{A}_{n, i_{k'}, j_{k'}, r_{k'}} \mathbf{x}_{t, \vec{i}} \frac{\partial L}{\partial \mathbf{y}_{\vec{j}}} \quad (3-5)$$

$$\frac{\partial L}{\partial \mathcal{G}_{n, \vec{r}}} = \sum_{\vec{i}} \sum_{\vec{j}} \prod_{k=1}^d \mathcal{A}_{n, i_k, j_k, r_k} \mathbf{x}_{t, \vec{i}} \frac{\partial L}{\partial \mathbf{y}_{\vec{j}}} \quad (3-6)$$

此外，我们还可以根据公式 (3-3) 推导出对应的导数公式 (3-7, 3-8)，便于我们

分析 forward 和 backward 过程的时间复杂度和空间复杂度。

$$\frac{\partial L}{\partial \mathcal{A}_n^{(k)}} = \frac{\partial L}{\partial \mathcal{Y}} \times_k \mathcal{X}_t \times_1 \mathcal{A}_n^{(1)} \times_2 \cdots \times_{k-1} \mathcal{A}_n^{(k-1)} \times_{k+1} \mathcal{A}_n^{(k+1)} \times_{k+2} \cdots \times_d \mathcal{A}_n^{(d)} \times_{1,2,\dots,d} \mathcal{G}_n \quad (3-7)$$

$$\frac{\partial L}{\partial \mathcal{G}_n} = \mathcal{X}_t \times_1 \mathcal{A}_n^{(1)} \times_2 \cdots \times_d \mathcal{A}_n^{(d)} \times_{1,2,\dots,d} \frac{\partial L}{\partial \mathcal{Y}} \quad (3-8)$$

3.3 模型参数量分析

因为我们保持了 RNN 其余部分的设计原则不变，仅改变了输入层模型，因此我们仅对输入层的参数量进行分析，同时讨论不同超参数设置下对模型参数的影响。如公式 (3-10) 所示，我们定义输入层的模型参数量为 P ，则原始矩阵乘法的参数量为 P_M ，BT-RNN 则为 P_{BTD} 。

$$P_M = IJ \quad (3-9)$$

$$P_{BTD} = N \left(\sum_{k=1}^d I_k J_k R + R^d \right) \quad (3-10)$$

其中 $\prod_{k=1}^d I_k = I$, $\prod_{k=1}^d J_k = J$ 。至于 P_M 和 P_{BTD} 的大小关系，我们根据不同超参数设置，计算并绘制了其对应的关系图，见图3-5。我们以原始的 RNN 模型为例，设置 $I = 4096$, $J = 256$, $N = 1$ ，此时原 RNN 模型拥有 $I \times J = 1,048,576$ 个参数。观察 Tucker 秩 R 和核心阶 d 对参数量的影响。根据公式 (3-10) 可知，当 d 值较小时，公式中的第一部分 $\sum_1^d I_k J_k R$ 主导了模型参数量；当 d 值较大时，第二项 R^d 主导了模型参数量。因此将看到参数量将随着 d 的递增而先减少，后迅速增加的过程（ $R = 1$ 时例外）。

然而为了达到减少参数的目的，我们可以通过超参数设置的方法，使得 $P_{BTD} < P_M$ ，且通常设置 $R \leq I_k, R \leq J_k, k \in [1, d]$ ，同时为了避免模型复杂度过高，核心阶通常也会设置较小，在我们的实验中， $d \in [2, 6]$ 较为合适。同时为了使得每一个维度权重尽可能均衡，会设置 $I_k \approx \sqrt[d]{I}$, $J_k \approx \sqrt[d]{J}$ 。因此整体而言，我们会通过超参数设置的方法，让模型参数量尽可能小，同时保持足够的表达能力。

3.4 模型复杂度分析

在 BT-RNN 模型的 forward 计算中，若是严格按照从左到右的顺序计算，公式 (3-3) 的峰值时间复杂度是 $\mathcal{O}(IJR)$ ，远大于原始矩阵乘法的复杂度。为此我们分析了其计算的中间变量结果，对其计算顺序做了调整，降低了中间变量的大小，从

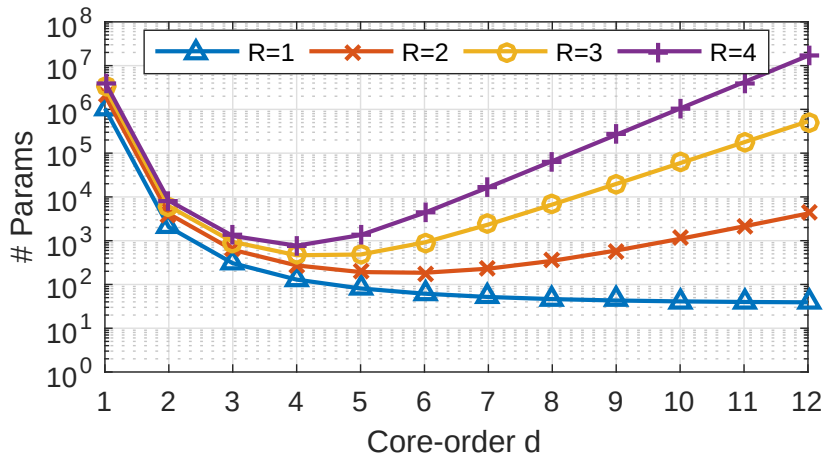


图 3-5 模型参数数量与秩的关系。图中坐标 x 轴表示核心阶 d ，即对应输入张量 $\mathbf{x}_t$ 的阶数；坐标 y 轴表示模型参数数量。这里展示的是 $I = 4096, J = 256, N = 1$ 时的情况。

而降低了时间复杂度。新的 BT 模型计算过程为：

$$\mathbf{y}_t = \sum_{n=1}^N \mathbf{x}_t \times_1 \mathcal{A}_n^{(1)} \times_2 \cdots \times_d \mathcal{A}_n^{(d)} \times_{1,2,\dots,d} \mathbf{g}_n \quad (3-11)$$

两者的区别是，公式 (3-3) 开始时沿着 R 维度做张量乘法，而在公式 (3-11) 中是沿着 I_k 维度的，而 $R \leq I_k$ ，因此后者的中间结果尺寸将更小，计算复杂度会更低一些。从而使得峰值时间复杂度从 $\mathcal{O}(IJR)$ 降低到 $\mathcal{O}(IJ_{\max} R^d)$ ，其中 $J = \prod J_k$ ， $J_{\max} = \max_k(J_k)$ ， $k \in [1, d]$ 。从而使得整个模型的计算复杂度从原来的 $\mathcal{O}(NdIJR)$ 降低到了 $\mathcal{O}(NdIJ_{\max} R^d)$ 。

表3-1列出了原始 RNN、TT-RNN^[62] 和 BT-RNN 的时间/空间复杂度，可以发现 TT-RNN 和 BT-RNN 都拥有较小的时间复杂度和空间复杂度，而 BT-RNN 由于计算过程中的中间变量较大，因此空间复杂度相比 TT-RNN 高一点。

表 3-1 三种不同的 RNN 模型复杂度分析。这里假设原始 RNN 的参数矩阵维度是 $I \times J$ ，张量化为 $I = I_1 \times \cdots \times I_d$ ， $J = J_1 \times \cdots \times J_d$ 。 $J_{\max} = \max_k(J_k), k \in [1, d]$ 。 TT-RNN 和 BT-RNN 设置为同样的秩 R 。

Method	Time		Memory	
	forward	backward	forward	backward
RNN	$\mathcal{O}(IJ)$	$\mathcal{O}(IJ)$	$\mathcal{O}(IJ)$	$\mathcal{O}(IJ)$
TT-RNN	$\mathcal{O}(dIR^2 J_{\max})$	$\mathcal{O}(d^2 IR^4 J_{\max})$	$\mathcal{O}(RI)$	$\mathcal{O}(R^3 I)$
BT-RNN	$\mathcal{O}(NdIR^d J_{\max})$	$\mathcal{O}(Nd^2 IR^d J_{\max})$	$\mathcal{O}(R^d I)$	$\mathcal{O}(R^d I)$

3.5 实验

RNN 是一个强大的序列处理工具，已被广泛用于计算机视觉领域和自然语言处理领域。在此我们设计了三个不同的任务实验：（1）视频分类任务，RNN 模型将直接面临尺寸庞大的图像输入，我们将在此检验 BT-RNN 在输入层的强大处理能力；（2）图片描述任务，即根据图片内容生成一段相符的描述句子，是一个典型的视觉和自然语言处理相结合的极具挑战性任务；（3）最后我们设计了一个重构实验，用来检验不同超参数设置对模型表达能力的影响，给出在不同场景下适用的超参数设置建议。

3.5.1 视频分类实验

视频分类任务其目的是根据视频内容的不同，把视频分成不同的类别。由于视频本身是一个在时间序列上的图像集合，因此通常使用 RNN 网络来处理。最常见的网络模型类别有两种，一种是端到端训练方式，即把每一帧图像当做输入数据，直接接入到 RNN 网络中。这种模型具有延迟低，计算开销小的优点，但是在精度方面略有不如。第二种方式是在 RNN 网络前接入一个预训练好的 CNN 网络，由 CNN 网络完成图像特征的提取工作，并由 RNN 网络完成分类任务。这种模型在精度方面有优势，但是由于 CNN 网络带来了极大的计算开销，导致模型在推理时延迟较高，具体应该根据应用场景和精度需求而定制模型。而由于在 RNN 前接 CNN 的模型基本不会面临输入层尺寸问题，这里我们仅以端到端训练方式为实验对象。

这里我们选用了 UCF11 动作识别数据集^[104]进行实验。UCF11 动作识别数据集是一个被广泛使用的视频分类数据集，它拥有 1160 个视频，被划分为 11 中动作类别，如：骑车，跳水，游泳，遛狗等。所有的视频都被压缩为 29.97fps 的 MPG 格式文件，所有视频的分辨率均为 $320 \times 240 \times 3$ 。为了方便处理，我们把图像帧缩放到 $160 \times 120 \times 3$ ，且每个视频随机采样 6 个关键帧作为序列输入数据。更详细的数据预处理内容可见文章 [65]。

在这里我们使用了一个 LSTM 模块作为基准模型，来对比测试 BT-LSTM 和 TT-LSTM 的优劣，模型结构展示如图 3-6 所示。需要注意的是，现在还有很多专注于提高食品分类精度的工作，如视觉注意力机制^[105]和时空模型^[106]等，这些工作与我们的工作相互独立的，在这里我们仅关注不同 LSTM 模型间的对比，与其他方法并不冲突。同时我们相信这些工作同样适用于我们的模型。

这里，对于基准模型的 LSTM，我们的参数设置是输入为 $I = 160 \times 120 \times 3 = 57600$ ，隐层大小为 $J = 256$ 。因此对于 LSTM 模型而言，输入层的参数量

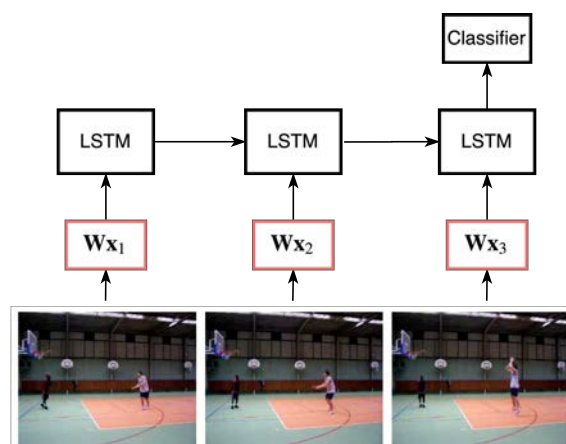


图 3-6 RNN-based 视频分类模型。

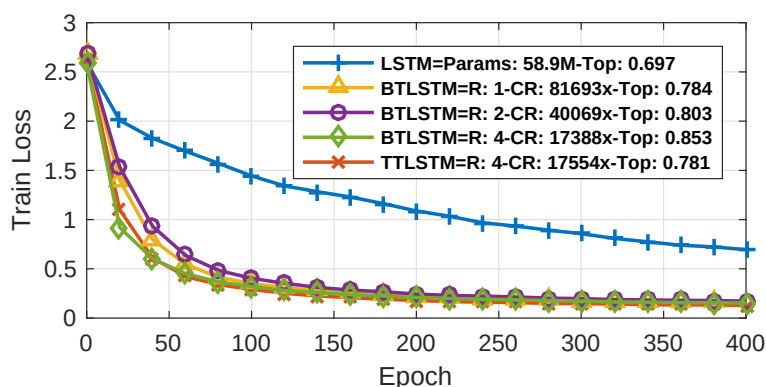
$P_M = 57600 \times 256 \times 4 = 58,982,400$ ，是一个极为庞大的参数矩阵。而对于 TT-LSTM 和 BT-LSTM，我们的参数设置是：核心张量的秩均为 $d = 4$ ，输入张量被张量化为 $I_1 = 8, I_2 = 20, I_3 = 20, I_4 = 18$ ，输入层的输出被张量化为 $J_1 = J_2 = J_3 = J_4 = 4$ ，TT-LSTM 的秩设置为 $R_0 = R_4 = 1, R_1 = R_2 = R_3 = 4$ ，BT-LSTM 的秩被设置成不同的值。

模型训练的收敛速度和精度请见图3-7。我们可以发现 BT-LSTM 的优点有：

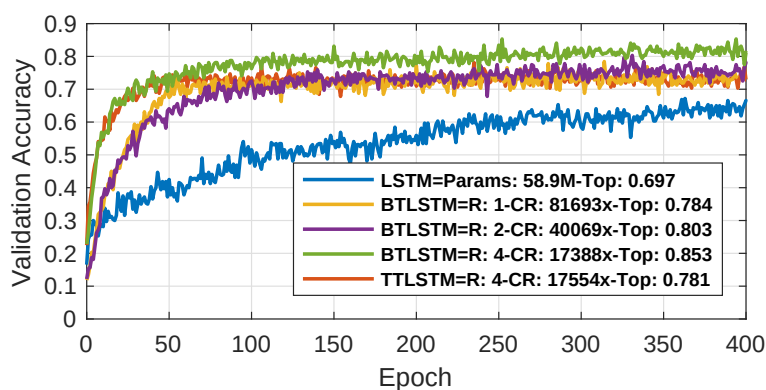
更小的模型尺寸 原始 LSTM 模型的输入层的参数量超过了 58.9M，然而 BT-LSTM 以更少的参数达到了更好的结果。根据公式 (3-10)，Tucker 秩为 1, 2, 4 时 BT-LSTM 的参数量分别是 721, 1470, 和 3387，对应的参数压缩比分别是 81693x, 40069x 和 17388x。

更快的收敛速度 从图3-7可以发现，在第 16 轮 (epoch-16) 迭代时 BT-LSTM 就达到了 60% 的测试集精度，而 LSTM 需要 230 轮才能达到同样的精度，在收敛速度上提升了 14 倍之多。即使在相似的模型参数量设置下，BT-LSTM-4 仍然比 TT-LSTM-4 的收敛速度要快 (见 [0, 50] 个 epoch 段)。

更好的模型测试精度 相比 LSTM，即使 BT-LSTM 节省了几个数量级的模型参数量，然而却获得了最高 15.6% 的精度提升，同时获得了相比 TT-LSTM 而言 7.2% 的精度提升，说明 BT 模型在表达能力方面有着良好的特性；同时我们观察到在不同秩的设置下，BT-LSTM 收敛到了相近的损失值，而增加秩会带来精度的提升，这与我们前文分析是一致的，更大的秩能够提升模型对数据特征的捕捉能力。我们在表3-2报告了目前在 UCF11 数据集上，端到端训练方式所能达到的最优精度



(a) 训练集损失值



(b) 测试集精度

图 3-7 UCF11 视频分类任务中不同网络结构的训练和测试情况。图中“CR”表示压缩比 (Compression Ratio)，R 是 BT 分解的 Tucker 秩，或 TT 分解的秩；“Top”表示最高的测试精度。

值结果。

良好的兼容性 此外，我们还在另一个被广泛使用的 RNN 模型，GRU 中对比测试了原始的 GRU 和 BT-GRU 模型的差异，结果如图3-8所示，可以发现其结果与 LSTM、BT-LSTM 的结论一致。一方面我们仅重构了 RNN 模型的输入层，对其余部分维持了其原本的设计思想，从而保证了新模型对已有模型的兼容性；另一方面也说明相比原始的全连接矩阵乘法而言，张量分解模型确实有着更优秀的特征捕捉能力。

3.5.2 图像描述任务实验

图像描述任务 (Image Captioning)，指的是给定一张图片，生成一段与内容相符的描述句子。是一个 CV (Computer Vision) 和 NLP (Natural Language Processing)

表 3-2 目前在 UCF11 数据集上端到端训练方式所能达到的最高精度值。

	Method	Accuracy
Orthogonal Approaches	Original [104]	0.712
	Spatial-temporal [106]	0.761
	Visual Attention [105]	0.850
RNN Approaches	LSTM	0.697
	TT-LSTM [65]	0.796
	BT-LSTM	0.853

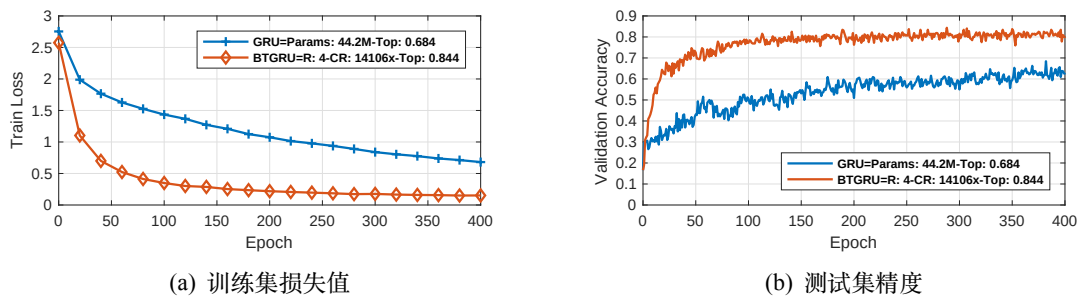


图 3-8 在 GRU 模型上的测试结果。

领域相结合的极具挑战性的任务。本文沿用了文章 [4] 所设计的模型，如图3-9所示。在结构上该模型是一个 CNN+LSTM 的结合，通过 CNN 模型进行图片特征提取，所提取到的图片特征作为 LSTM 的初始输入，然后把真实的句子按序列顺序输入到 LSTM 中，让 LSTM 预测下一时刻可能的单词。在推理时，则使用 BeamSearch 的方式生成一系列候选单词。

这里 LSTM 的输入层将面临图像的特征向量和单词的嵌入向量两种迥然不同的数据，而且由于图像特征向量通常维度较大，因此我们将把 LSTM 的输入层替换为我们的 BT 模型，测试 BT-LSTM 在图像描述任务中的适用性。在这个实验中我们使用了 MSCOCO^[107] 数据集，一个用于目标检测、图像分割和图像描述的大型数据集，训练集包含了 82783 张图片，而测试集则包含了 40775 张图片。针对每一个图片的预处理过程，我们依照原文的描述，将图片缩放到 $224 \times 224 \times 3$ ，并且每个像素减去对应的 channel 均值 (103.939, 116.779, 123.68)。而 CNN 网络我们使用了 Inception-V3 网络。

表3-3展示了在 MSCOCO 数据集上不同模型的表现情况，这里我们使用了 BLEU-4^[108] 和 CIDEr^[109] 指标。可以发现，在这个模型上 BT-LSTM 对比 LSTM 的优势效果不太明显，当 $N = 1$ 时（退化为 Tucker 分解）其精度甚至不如原始的 LSTM，这是由于一方面图像已经经过了 CNN 网络，提取出了足够紧凑的特征向

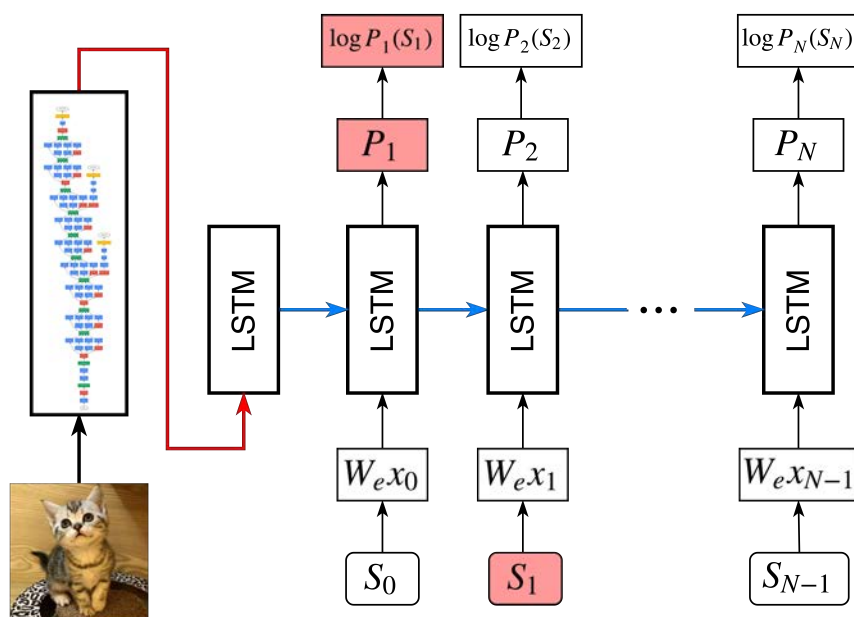


图 3-9 图片描述模型。

表 3-3 MSCOCO 的数值结果对比，其中 TT-LSTM 和 BT-LSTM 的超参数设置均为：
 $d=4$ ， $R=4$ 。BLEU-4 和 CIDEr 指标越大越好。

Method	#Params	BLEU-4	CIDEr
Original ^[4]	1048576	0.277	0.855
LSTM (reproduced)	1048576	0.299	0.910
BT-LSTM N=2	2048	0.307	0.921
TT-LSTM	1152	0.291	0.892
BT-LSTM N=1	1024	0.296	0.907

量，同时输入单词也经过了词嵌入（Embedding）处理，因此此时 LSTM 的输入层将面临较为紧凑的输入向量，输入层的参数矩阵低秩特性并不明显，因此 BT 模型带来的提升空间有限。然而我们发现，BT-LSTM 当 $N=2$ 时，仍然获得了小幅度指标的提升，而此时的参数量远小于 LSTM，一方面说明了 BT 模型相比单一的 Tucker 模型有着更为强大的特征捕捉提取能力，另一方面也说明了原始的参数矩阵仍然有大量的冗余，BT 模型在此场景下仍然有效。而作为对比，TT-LSTM 结果则较差，即使以相似的参数量构建模型，其指标仍然差于 BT 模型。图3-10展示了部分图片的描述句子生成情况，可见两个模型都能生成足够通顺和描述较为准确的句子，而 BT 模型明显有着更准确的表述。

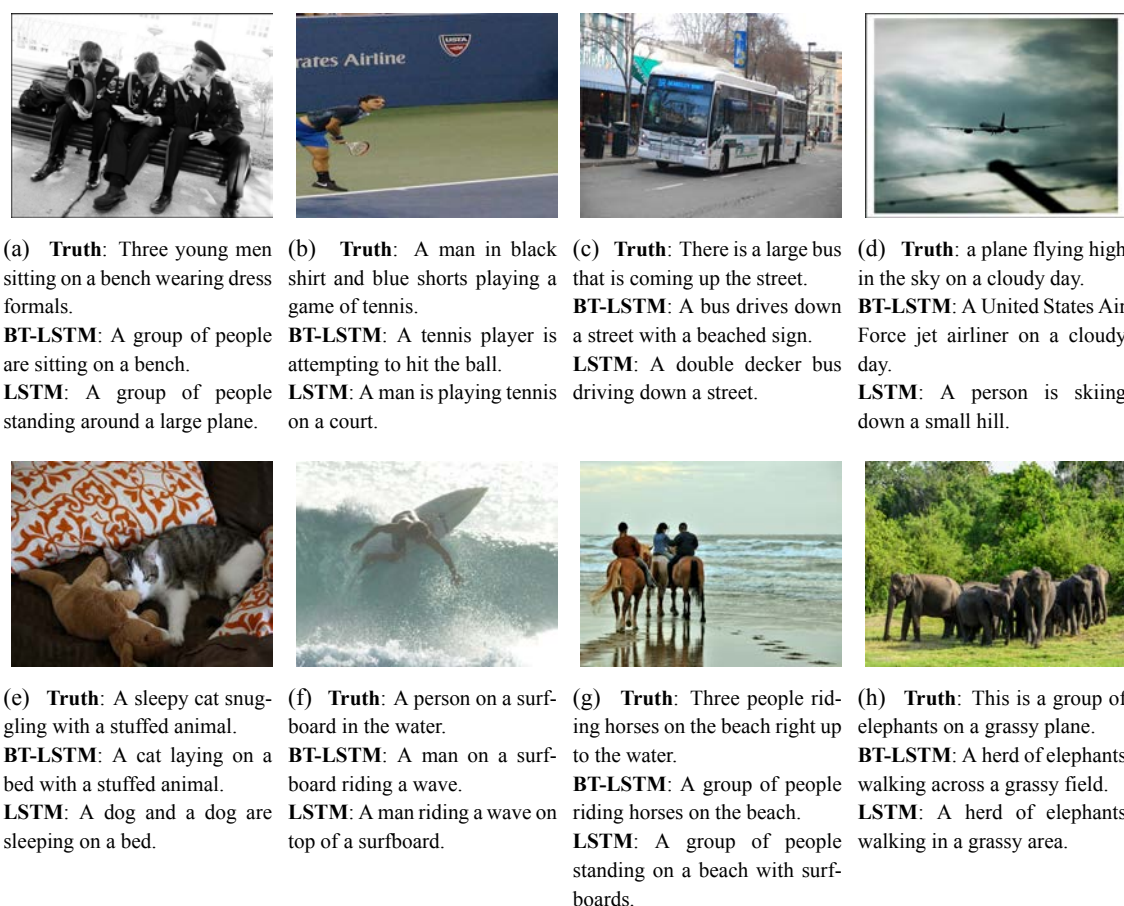
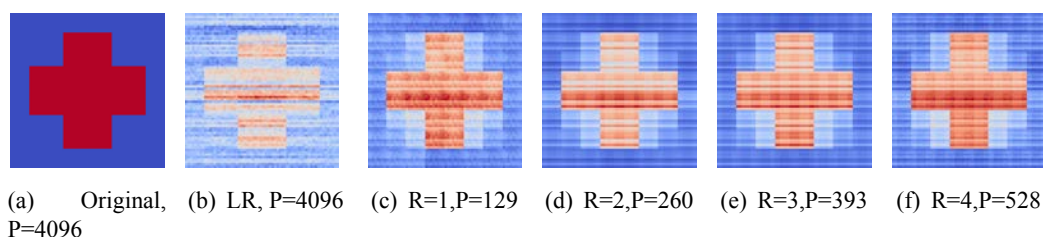
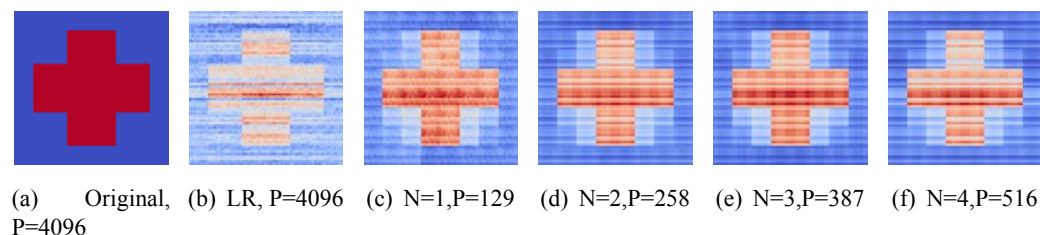


图 3-10 MSCOCO 数据集的图像描述结果展示。

3.5.3 参数选择实验

为了获得不同超参数设置对模型影响的情况，以及对超参数设置给出建议，我们设置了一个简单的重构实验，从另一个角度观察 BT 分解与矩阵分解在特征捕捉能力方面的异同。

在 BT-LSTM 模型中，有三个关键超参数，即模型的 Tucker 秩 R ，核心张量的阶 d ，以及 CP 秩 N 。我们构造了一个简单的线性模型 $\mathbf{y} = \mathbf{W}' \cdot \mathbf{x}$ ，其中 $\mathbf{W}'$ 是预先设置的， $\mathbf{x}, \mathbf{y} \in \mathbb{R}^{64}$ ， $\mathbf{x}$ 从一个高斯分布 $\mathcal{N}(0, 0.5)$ 中采样。 $\mathbf{y}$ 则是可以根据 $\mathbf{x}$ 和模型计算得出。现在我们得到了训练数据 $\mathbf{x}$ 和 $\mathbf{y}$ ，然后构建一个新的线性模型 $\mathbf{y} = \mathbf{W} \cdot \mathbf{x}$ ，用得到的训练数据（需要加上一个小噪音，以防止过拟合）训练新的模型。由此可知，新的模型参数 $\mathbf{W}$ 应该尽可能接近 $\mathbf{W}'$ 。请注意这个实验仅仅是为了展示不同参数设置下模型参数的恢复能力，并没有加入如 L1 正则化和 Lasso 正则化等方法。

图 3-11 Tucker 秩 R 的选择。其中 $d=2$, $N=1$ 。图 3-12 CP 秩 N 的选择。其中 $d=2$, $R=1$ 。

Tucker 秩 R 的选择 图3-11展示了在超参数设置 $d = 2$, $N = 1$ 时不同的 Tucker 秩对参数的恢复能力, P 表示模型参数量。其中图3-11(a)表示原始模型参数 $\mathbf{W}'$, 图3-11(b)是直接使用矩阵的线性回归方法恢复的图形。可见随着 Tucker 秩 R 的逐渐增加, 模型恢复能力越强。而在面临带噪音的输入数据时, BT 模型明显具有更优秀的抗干扰能力, LR 模型则受噪音影响严重。

CP 秩 N 的选择 图3-12展示了不同的 CP 秩 N 的差异情况。可以发现, 当 $N > 2$ 时, 对比 $N = 2$ 的图形并不能获得太大的提升, 然而对比 $N = 1$ 时明显噪音少了, 说明单一的 Tucker 模型在抗噪音方面略有不足, 在模型训练时可能会带来表达能力欠缺, 模型收敛不好的问题。而 N 过大会导致模型参数量和计算复杂度的上升, 因此选择一个合适的 CP 秩尤为重要。

核心张量阶 d 的选择 图3-13展示了不同的核心阶 d 的差异情况。从图中可以发现, 当 $d = 2$ 时模型恢复能力最好, 而随着 d 的增加, 参数量随之增加, 然而却几乎无法恢复出参数矩阵, 可见 d 的选择尤为关键。这里是由于原始参数矩阵是一个较为特殊的低秩矩阵, 当 d 过大时, 过高的模型复杂度反而不适合低秩结构模型。

3.6 本章小结

在面临高维数据输入时, 传统的 RNN 网络将出现输入层参数量过大, 模型复杂度过高, 训练时需要消耗大量的硬件资源。在此我们提出了 BT-RNN 网络, 以

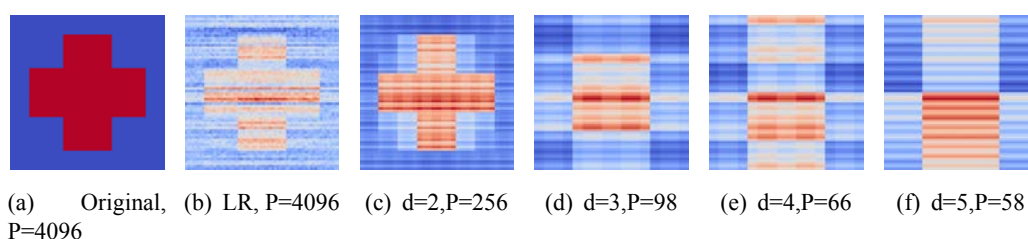


图 3-13 核心阶 d 的选择。其中 $R=1$, $N=2$ 。

低秩 BT 分解替代传统的参数矩阵，达到了降低参数量的同时降低了模型复杂度，缓解了模型过拟合问题，从而获得了精度上的提升。我们同时在 LSTM 和 GRU 两个重要的 RNN 变种模型中完成了实验，实验表明在面临如图像数据，BT 模型能获得极快的收敛速度和极高的收敛精度，同时达到了端到端训练方式的目前最佳结果。而在图像描述任务中，输入层面临的不再是单一的图像数据，而是紧凑的图像特征向量和单词嵌入向量，虽然我们不能再简单的做出低秩假设，但是 BT 模型仍然工作良好，获得了更好的精度，同时大幅度降低了模型参数量。因此，相比传统的 RNN 模型和 TT 版本的 RNN 模型，BT 模型有着更优秀的表达能力，能以更少的参数量达到甚至超过以往的精度，同时大幅度加快了模型收敛速度。

与量化和低精度计算方法相比，BT-RNN 模型能在任何计算设备上运行并获得计算效率和模型精度的提升；然而因为仅针对了低秩的输入层进行优化，因此加速效果不如量化和低精度方法。此外，与知识提取方法相比，BT-RNN 模型无需一个良好的“教师”模型，同时在不同任务中均能获得较大的精度提升；然而由于输入层的低秩分解特性，因此仅能处理高维低秩的输入数据，如图片等，而非高维低秩，如文本数据时，BT-RNN 模型表现较差。

第四章 动态内存管理技术

本章将从工程实现的角度解决卷积神经网络在训练过程中的内存管理问题，尝试在工程实现的层面上降低训练过程的内存压力，使得训练更大的模型成为可能。本章将介绍三个关键的动态内存管理技术，并在实验中证明这些技术的有效性，展示我们的实现在训练速度、模型大小方面的优势。需要注意的是，本章所讨论的内存管理仅限于在计算过程中产生和消耗的中间变量（如每一层的输入和输出变量），对于模型参数我们默认常驻于设备内存中不做处理。

4.1 变量声明

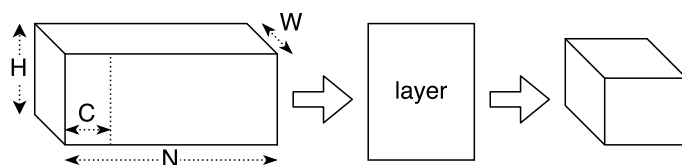


图 4-1 张量，即计算过程中的一个 $N \times C \times H \times W$ 多维数组。其中 N 表示批大小 (batch-size)， C 表示通道数 (Channel)， H 和 W 表示一个特征图 (Feature Map) 的高 (Height) 和宽 (Width)。

为了描述 CNN 的内存使用情况，我们沿用张量 (tensor) 表示计算过程中的变量，此时的张量仅表示一个 $N \times C \times H \times W$ 的多维数组，如图4-1。我们用 l_i^f 表示第 i 层 forward 时的内存占用，如图2-16中的输入变量 x^l 和输出变量 x^{l+1} ； l_i^b 表示 backward 时的占用，如反传的误差项 ∂f^{l+1} ，传回到上一层的误差项 ∂f^l ，参数梯度 $\nabla \omega^l$ 等；用 $peak_m$ 表示一次 forward-backward 过程中的峰值内存占用，也可理解为整个网络训练过程中所必须的内存开销。为了展示不同技术对内存优化的情况，我们以简单的全分配策略作为对比的基准，即每一个独立的张量将被分配独立的内存，此时 $peak_m = \sum_{i=1}^N (l_i^f + l_i^b)$ ；我们用 $l_{peak} = \max(l_i)$ 表示单层的最大内存占用。为了统一 forward-backward 过程，我们将过程定义为 $2N$ 步，前 N 步完成 forward，后 N 步完成 backward，如图4-2所示；定义 $R_k, k \in [1, 2N]$ 表示每一步计算所需的需张量， $t_k, k \in [1, 2N]$ 表示每一步的计算输出张量。

4.2 张量生存期分析

张量生存期分析技术 (Liveness Analysis) 来源于编译器技术中的变量生命周期管理技术，即追踪每一个变量的生命周期，在合适的时间点回收资源。这里我

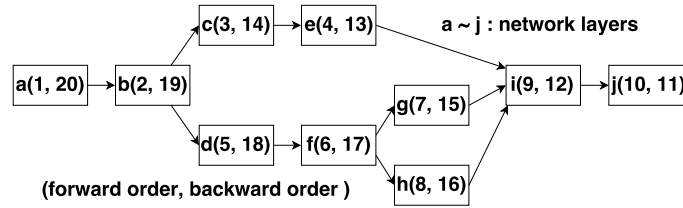


图 4-2 非线性结构网络的执行顺序。图中共有 $N = 10$ 层，总计算步数为 20 步，包括了前 10 步的前向计算和后 10 步的反向传播。

们针对在单次 forward-backward 过程中张量的使用情况。其原理是分析每一个张量在单次 forward-backward 的使用时间的起点和终点，仅在起点处申请分配内存，在终点处回收资源；在下次 forward-backward 迭代时重复以上操作。理论分析此技术能节省约 50% 的内存占用。目前基于计算图实现的深度学习训练框架如 Tensorflow^[72]、PyTorch^[76]、MxNet^[75] 等均实现了自动求导规则，隐藏了求导过程的细节。但为了描述张量间的依赖关系，这里仍以传统的数据流形式展示张量间的依赖，相关技术可以迁移到上述平台上。

为了描述张量的前后依赖关系，我们用 in 集合表示每一步在计算前存活的张量；用 out 集合表示每一步计算后仍然存活的张量。可用时间复杂度为 $\mathcal{O}(N^2)$ 的算法记录获得每一步的 in 集合和 out 集合，具体见算法 1。

Algorithm 1: in/out 集合构造

Data: each step the require tensors R_k ; each step the output tensor O_k

Result: in/out set of each step

```

1 for  $i \leftarrow 1$  to  $2N$  do
2    $flag = False$ ;
3   for  $j \leftarrow 2N$  to  $i + 1$  do
4     if  $t_i \in R_j$  then  $flag = True$ ;
5     if  $flag == True$  then
6        $in[j].insert(t_i)$ ;
7        $out[j].insert(t_i)$ ;
8     end
9   end
10 end

```

通过对比每一步的 in 集合和 out 集合，我们就可获知在每一步计算结束后可以被安全释放的张量。图 4-3 展示了在单次 forward-backward 过程中每一步的 in/out 集合情况。具体来说，假设当前已完成 FC 层的 backward 计算，通过对比 FC 层的 in/out 集合我们知道张量 t_2, t_3, t_6 不在 out 集合中，意味着在当前 forward-backward 过程中这两个张量不再使用，因此可以安全的释放并回收这部分资源。

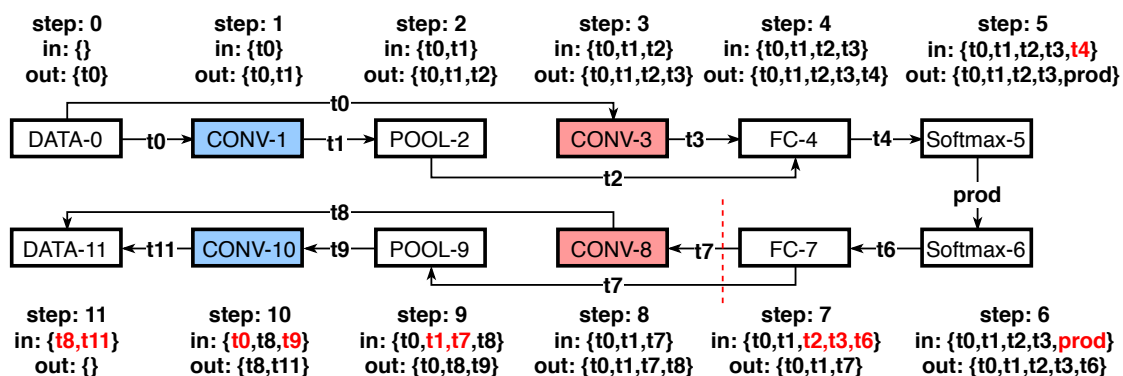


图 4-3 in-out 集示意图。其中红色标记的变量不在对应的 out 集合中，表示这些变量可在当前层被安全释放。

理论而言，张量生存期分析技术可以把峰值内存占用降低到 $\sum_{i=1}^N l_i^f + l_N^b$ 。为了简化分析过程，我们假设每一层 forward 和 backward 时的内存占用一样，即 $l_i^f = l_i^b$ ；同时假设在 forward 过程的输出张量将被用于计算梯度^①。因此在 forward 过程中 step- k 时的内存占用是 $cost_k^f = \sum_{i=1}^k l_i^f, k < N$ ；在 backward 阶段的 step- k 时，张量生存期分析技术会释放所有的 l_i^f 和 l_i^b ，其中 $i \in [k+1, N]$ ，如图4-4，此时内存占用为 $cost_k^b = \sum_{i=1}^k l_i^f + l_k^b$ 。由此可知在 forward 结束并计算了最后一层的 backward 时占用的内存达到峰值， $peak_m = \sum_{i=1}^N l_i^f + l_N^b$ 。因此，通过张量生存期分析技术可节省约 50% 的内存开销。

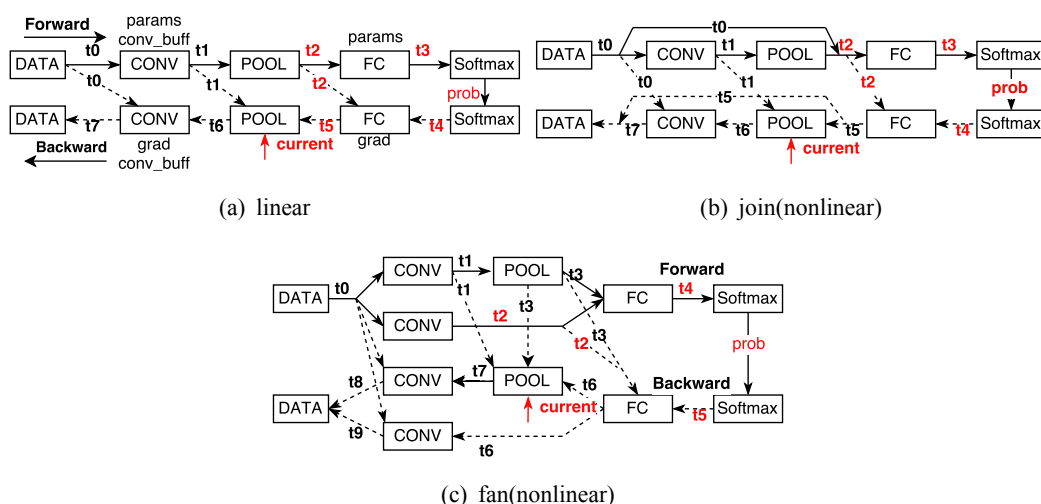


图 4-4 三种主流的网络结构上的应用张量生存期分析后变量的存活情况。其中红色的变量表明这些张量已被释放，或可被安全释放。

① 计算梯度时并非所有层都需要输出张量，这里为了简化分析而假设。

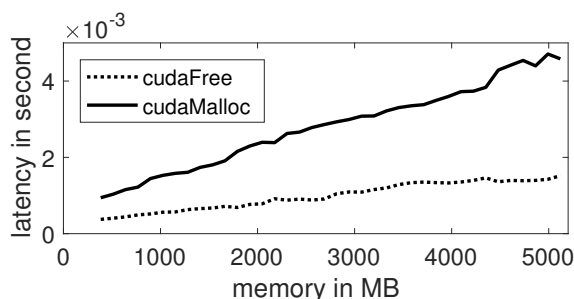


图 4-5 cudaMalloc 和 cudaFree 操作耗时。当开启了变量生存期分析技术后，在网络单次 forward-backward 过程中，将会涉及到上百甚至上千次的内存申请和释放操作，占比比例极高。

4.3 统一的内存池技术

由于网络结构在训练过程中通常保持不变，因此网络每一层的 in/out 集合可在网络构建完成后执行一次即可，所产生的计算开销可忽略不计。然而每一轮 forward-backward 过程都引入了大量的内存分配、释放操作，而网络通常需要数十万甚至上百万次迭代才能训练收敛，由此带来了极其频繁的内存分配及释放操作，严重影响了系统性能。经测试发现，在 ResNet50^[37] 网络训练过程中，cudaMalloc 和 cudaFree^①耗时占了整体耗时的 36.28%。图4-5展示了单次申请和释放 GPU 内存的耗时。

我们观察到网络结构和张量的尺寸在训练过程中保持不变，因此可以引入内存池技术，解决上述问题。同时考虑到某些深度网络在训练过程中可能占用超过 100GB 的内存，GPU 的片上内存远远不能满足需求。进一步地，我们提出了统一的内存池技术 (Unified Tensor Pool, UTP)，利用异步数据传输，统一管理 CPU 和 GPU 内存。具体而言，即当 GPU 内存不足时，可以把部分数据传输到 CPU 内存上，缓解 GPU 上的内存压力；当 GPU 内存压力较小时，把 CPU 内存上的数据调回 GPU。

4.3.1 数据卸载与数据预取

然而并非所有的张量都适合被透明地从 CPU 和 GPU 端迁移。由于 CPU 与 GPU 间的 PCI-e 数据通道传输速度远不如 GPU 的计算速度，因此需要合理规划需要搬移的张量。如图4-6所示，在 CNN 网络中，卷积层普遍占据了 30% 以上的内存，同时占据了 50% 以上的训练时间，我们将它归为 checkpoint 层；如池化层 (POOL) 和激活层 (ACT) 占据了较大的内存，然而计算开销占据极小，搬移这部

① CUDA 中申请和释放 GPU 内存的函数。

分张量会极大影响训练速度。因此 UTP 内存池仅对卷积层张量进行透明搬移，池化层、激活层等的处理方法将在本文后续介绍。

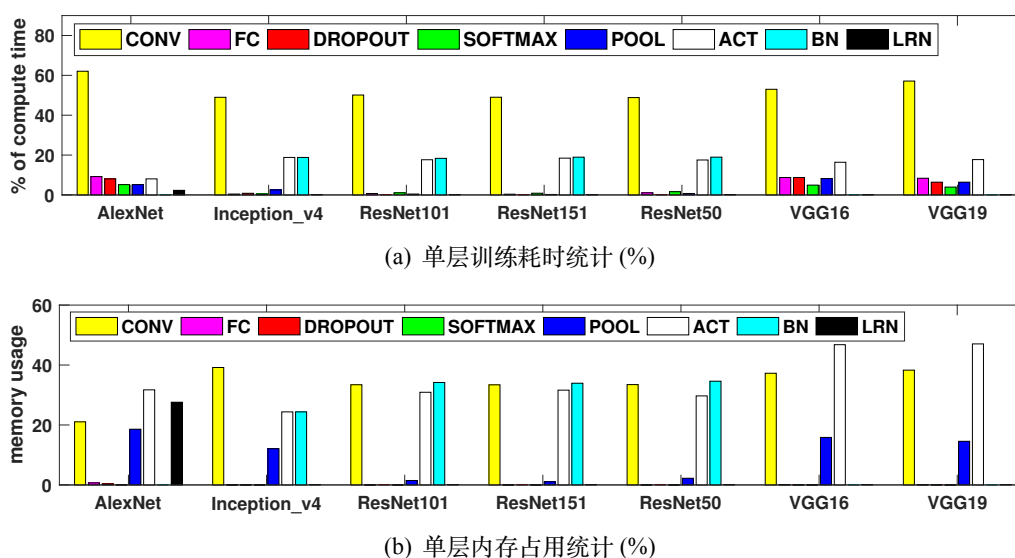


图 4-6 单层数据统计情况。训练耗时包括了 forward 和 backward 耗时。

4.3.2 Tensor Cache

由于网络结构固定，因此我们可以预测并规划出每一层当前的内存占用和后续层计算的内存占用情况，提前对卷积层张量卸载到 CPU 内存或从 CPU 内存预取到 GPU 内存上。同时观察发现，forward-backward 过程是一个从头到尾、从尾到头的过程，其中在头部靠前的层的张量将在未来最后被用到；在尾部靠后的层的张量将会在 backward 最先被使用，近似符合“最近最少使用”替换原则（Least Recent Used, LRU），如图4-7。当预测到 GPU 内存不足时，减小 LRU 队列，把尾部张量搬移到 CPU 内存上；当预测到下一层所需张量位于 CPU 内存时，把数据从 CPU 内存搬移到 GPU 上。而当前层所使用的张量将会被插入到 LRU 队列头部，表示最近被使用过。具体过程见算法2的描述。

4.4 代价最优再计算策略

在前文中提到，POOL、ACT、LRN 和 BN 层计算耗时总体不超过 10%，然而却占据了约 50% 的内存，那么在极端内存需求的情况下，“时间换空间”不失为一个可行的方案。在 MxNet 平台上实现了一个“再计算策略”（re-computation）^[110]，具体而言是在网络中设置某些层作为 checkpoint，checkpoint 层的变量常驻在 GPU 上，非 checkpoint 层变量在面临 GPU 内存不足时会被释放，当需要使用时

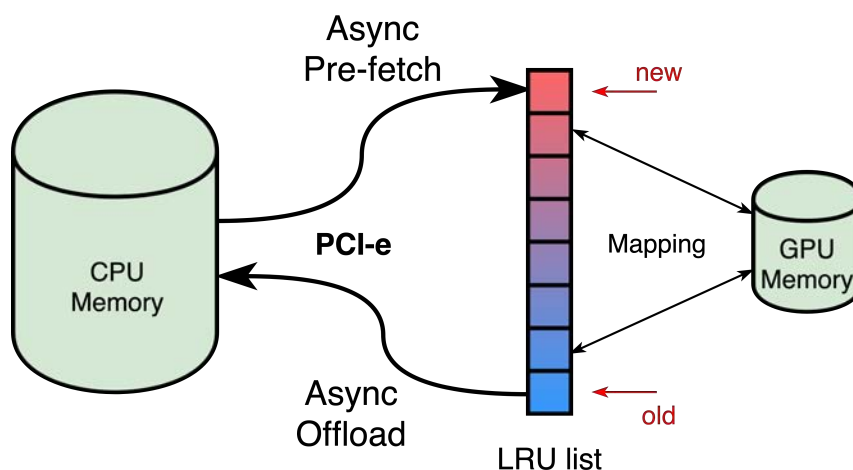


图 4-7 基于 UTP 统一内存池构建的 Tensor Cache 缓存策略。

从 checkpoint 处再算一遍 forward，获得所需变量。后来在 PyTorch1.0 版本中亦有一个类似的实现^①。

我们分析了两者的实现，发现均未能在训练速度和内存消耗方面达到最优的均衡，因此提出了最优代价再计算策略（cost-aware re-computation）。具体而言，我们关注 checkpoint 与 checkpoint 之间这一段网络的内存释放分配情况，结合图4-8，将再计算策略分为三种实现方式：

- 速度最优策略：当发生内存不足时将会把一整段网络的变量全部释放，当面临张量缺失时从 checkpoint 处重新 forward 一遍，并把所有中间张量保存下来，整体内存消耗是 $l_1^f + l_2^f + l_3^f + l_4^b$ ，再计算策略时间复杂度是 $\mathcal{O}(N)$ ；
- 内存最优策略：当发生内存不足时将会把一整段网络的变量全部释放，当面临张量缺失时从 checkpoint 处重新 forward 一遍，并把所有中间张量全部释放，整体内存消耗是 l_4^b ，再计算策略时间复杂度是 $\mathcal{O}(N^2)$ ；
- 代价最优策略：当把网络中的层标记为 checkpoint 后，会将网络切分成若干段；我们可以提前计算出最大内存占用的层 l_{peak} ，若某一段内存的整体占用不超过 l_{peak} ，则应用速度最优方式；否则应用内存最优方式。在实践中，最优代价方式的内存消耗与内存最优方式相当，而训练速度与速度最优方式相当。

我们将通过分析这三种实现策略下模型的峰值内存占用来证明我们提出的最优代价策略的性能。由于所有 checkpoint 层的内存占用固定不变，为了简化分析，省去此部分内存占用；假设当前段内存整体消耗为 $cost_{seg} = \sum_k^{seg} (l_k^f)$ ，所有非 checkpoint 层均被释放，当前计算位于某一段的 backward 阶段，当前层为 l_i^b ：

^① <https://pytorch.org/docs/master/checkpoint.html>

Algorithm 2: UTP 及 LRU 的使用

Data: each step the require tensors R ; LRU ; UTP pool P ; Prefetching offset s

```

1 Function Unload( $i$ ):
2   while  $GPU.memUsed + \sum_{k=i}^{i+s} R_k > GPU.memSize$  do
3      $t \leftarrow LRU.PopLast()$ ;
4      $P.AsyncGPUtoCPU(t)$ ;
5      $GPU.memUsed = GPU.memUsed - t.size$ ;
6   end
7 Function Prefetch( $i$ ):
8   if  $R_{i+s}$  not in  $GPU$  then
9      $P.AsyncCPUtoGPU()$ ;
10  end
11 Function Training():
12  for  $i \leftarrow 1$  to  $2N$  do
13     $LRU.insertHead(R_i)$ ;
14    Unload( $i$ );
15    Prefetch( $i$ );
16    if  $i \leq N$  then forward( $i$ ) else backward( $i$ );
17  end

```

- 速度最优策略: $cost = cost_{seg} + l_i^b$; 若 l_{peak} 位于当前段, 则 $cost > l_{peak}$; 否则需要对比 $cost_{seg}$ 与 l_{peak} 的关系; 因此整体而言 $cost \geq l_{peak}$;
- 内存最优策略: $cost = \max(l_k^f) + l_i^b$; 可得 $cost \leq l_{peak}$;
- 代价最优策略: 若 $cost_{seg} \leq l_{peak}$, 应用速度最优策略, $cost = cost_{seg} + l_i^b$, 此时 $cost \leq cost_{seg}$; 否则应用内存最优策略, $cost = \max(l_k^f) + l_i^b$, 则 $cost = l_{peak}$; 因此整体 $cost \leq l_{peak}$ 。

我们通过实验统计了这三种不同策略的峰值内存情况和需要额外 forward 计算的
网络层数, 请见表4-1。

表 4-1 三种不同的再计算策略。其中 extra 栏表示需要额外 forward 计算的
网络层数。

可以看到, 我们提出的代价最优策略在峰值内存上与内存最优策略相当, 而
额外计算的层数与速度最优策略相当。

Network	Speed-aware		Memory-aware		Cost-aware	
	extra	$peak_m(\text{MB})$	extra	$peak_m(\text{MB})$	extra	$peak_m(\text{MB})$
AlexNet	14	993.018	23	886.23	17	886.23
ResNet50	84	455.125	118	401	85	401
ResNet101	169	455.125	237	401	170	401

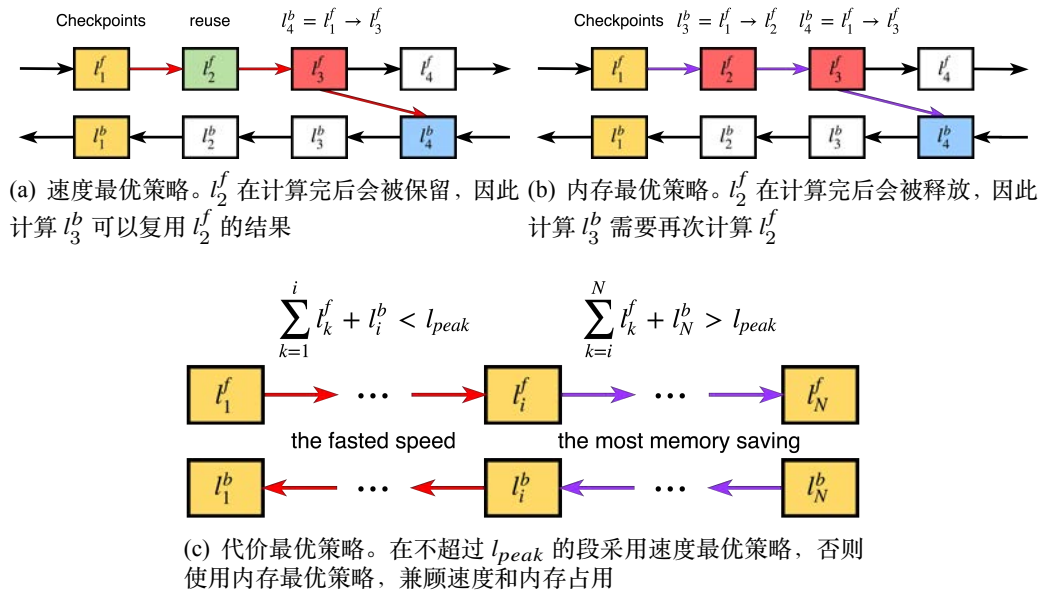


图 4-8 再计算策略的三种实现方式。

4.5 动态卷积选择算法

不同的卷积实现算法对运算速度影响极大，常用的算法如 FFT、Winograd 和 GEMM 等，如 CUDNN^[80] 计算库就包含了 7 种不同的卷积实现算法。这些不同的实现在运算速度和 buffer 空间上大致呈正比例关系，即速度越快的算法消耗越大的 buffer 空间。基于我们的动态内存分配策略，网络的每一个卷积层的可用内存将难以提前确定，因此我们实现了一个动态的卷积选择算法，用以确定每一个卷积层在可用内存受限情况下速度最快的卷积实现。如图4-9所示，当卷积的 Buffer 空间充足时，可获得极大的加速比。

4.6 实验

为了准确测试我们提出的三个关键动态内存管理技术以及这些技术对最终模型训练的影响，我们把实验分为两部分完成：第一部分在 baseline 的基础上逐步添加这三个技术，观察内存使用情况是否符合预期；第二部分整合这些技术后，在常用的 CNN 网络中与其他主流训练平台的测试对比。

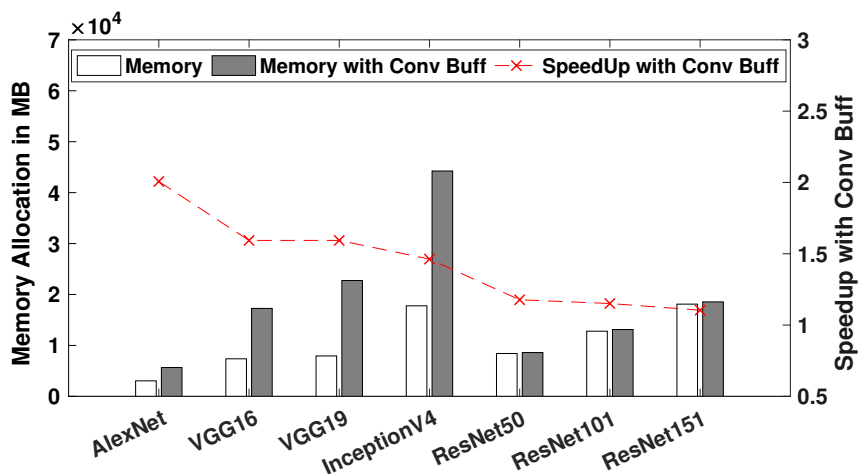


图 4-9 当卷积的 Buffer 空间充足时,可获得最高达 2 倍的加速。图中左侧坐标轴表示网络的总体内存消耗;右侧表示在开启 Buffer 支持后训练的加速比;AlexNet 的 batch size 是 200,其余网络为 32。

4.6.1 单项测试

我们以一个 23 层的 AlexNet^①为例进行试验。假设 batch-size 设置为 200,此网络的 baseline 模型需要计算 23 层的 forward 和 backward,申请 36 个张量,共消耗 2189.437MB 内存。

4.6.1.1 张量生存期分析测试

当使用了张量生存期分析后,最多存活的张量为 17 个,峰值内存消耗为 1489.355MB,节省了 31.9% GPU 内存。而整个 forward-backward 过程的张量个数变化和内存变化如图4-10所示,其中峰值内存出现在第 32 步,即 POOL5 的 backward。前文中我们为了简化计算,假设了网络每一层的内存消耗相等,因此得出了峰值出现在第一个 backward 层。然而真实的网络模型很难做到每一层都均衡,因此会出现峰值迁移的情况。

4.6.1.2 数据卸载和预取测试

数据卸载与数据预取理论上能把峰值内存降低到 $\sum_{i=1}^N (l_i^f \notin checkpoints) + l_N^b$ 。我们把 CONV 层设置为 checkpoint 层,将被用于数据卸载和预取,其余层保持在 GPU 上常驻,回退到应用张量生存期分析技术上。如图4-11所示,新的峰值内存出现在第 39 步,POOL2 的 backward 中,为 1132.155MB,对比张量生存期

① CONV1 → RELU1 → LRN1 → POOL1 → CONV2 → RELU2 → LRN2 → POOL2 → CONV3 → RELU3 → CONV4 → RELU4 → CONV5 → RELU5 → POOL5 → FC1 → RELU6 → Dropout1 → FC2 → RELU7 → Dropout2 → FC3 → Softmax

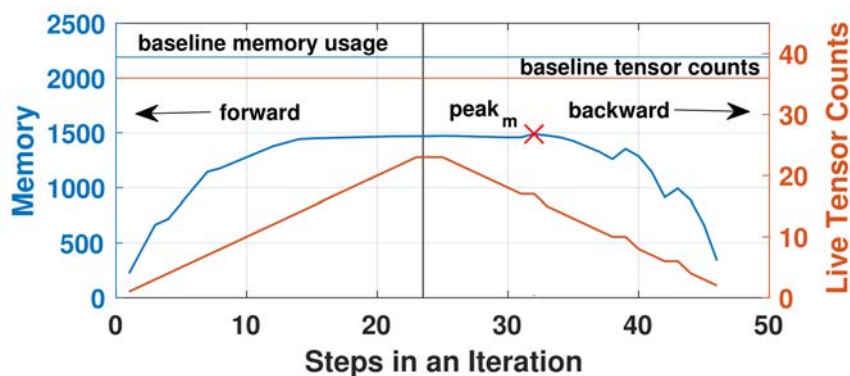


图 4-10 张量生存期分析。图中中轴线的左侧为 forward 过程，右侧为 backward 过程。

蓝色线表示内存占用变化情况，黄色线表示张量在 GPU 上存活的数量变化。

分析降低了 357.2MB，对比 baseline 提升了 48.29%，这里峰值内存从 POOL5 迁移到了 POOL2。CONV1 到 4 的内存分别是 221.56MB，142.38MB，49.51MB 和 49.51MB。当在 POOL5 的 backward 过程中，此时 CONV1-4 均被卸载到了 CPU 内存上，GPU 内存此时为 $1489.355 - 221.56 - 142.38 - 49.51 = 1075.9\text{MB}$ ，明显小于 POOL2 的 backward 开销，因此出现了峰值迁移现象。而此时，CONV1 被卸载，而 CONV2 将被预取，因此 POOL2 的 backward 时存活张量数应该是 $10(\text{张量生存期分析中的测量值}) - 1(\text{CONV1}) = 9$ 个。

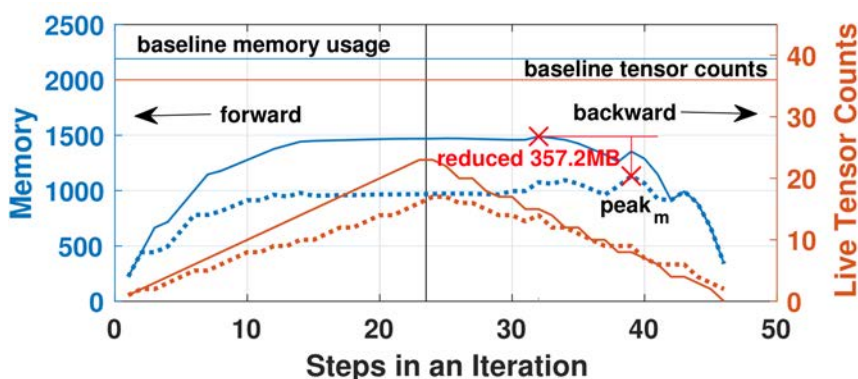


图 4-11 数据卸载/预取策略。在张量生存期分析技术的基础上再降低了 357.2MB 的峰值内存占用，同时峰值出现更晚了一些。

4.6.1.3 代价最优策略测试

在上述两个技术的基础上，我们再加上代价最优的再计算策略，如图4-12所示，峰值内存继续减少到 886.385MB。理论上，此三项技术的内存峰值应为单层内存的最大值，而 $\max(l_i) = 886.385\text{MB}$ ，出现在 LRN1 的 backward 计算中，即第 44 步。图中的峰值情况确实符合预期。最终，我们把内存占用从最初的 2189.437MB

降低到了 886.385MB，仅为原来内存的 40.5%，效果显著。

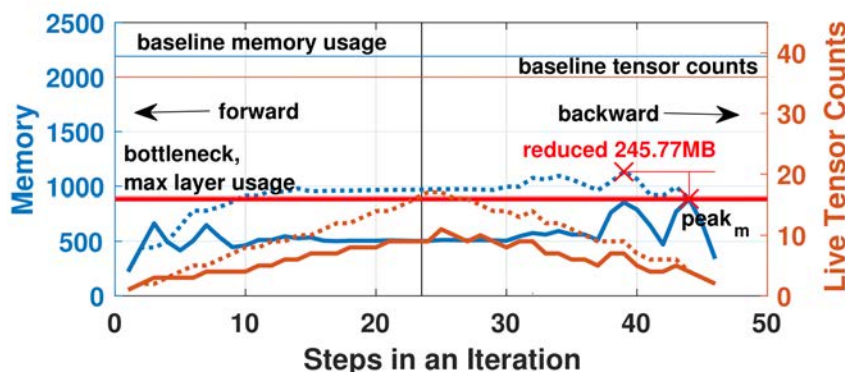


图 4-12 代价最优再计算策略。在此前两个技术的基础上再次降低 245.77MB 的峰值内存占用，峰值出现再次延后，这是由于越靠近输入端的层占用的内存越大导致的。

4.6.1.4 内存池性能测试

前文提到，为了弥补频繁申请、释放内存带来的性能损耗，我们设计了硬件层面统一的内存池。由于张量生存期分析会大幅度降低存活的张量个数，因此我们将只开启张量生存期分析技术，用于测试在频繁内存申请和释放时，内存池的性能表现。其结果如表4-2所示。可见在线性结构的网络如 AlexNet 和 VGG16 中，由于其本身张量个数不多，因此内存池带来的性能提升有限；而如 ResNet152 这种具有大量张量的网络，就可以发现内存池的性能提升极大。可见随着张量个数上升，内存池的性能表现将会更有优势。

表 4-2 内存池性能对比。其中 CUDA 项表示使用标准的 `cudaMalloc` 和 `cudaFree` 操作。AlexNet 的 batch-size 为 128，其余为 16。测试平台为 NVIDIA K40。

img/s	AlexNet	VGG16	InceptionV4	ResNet50	ResNet101	ResNet152
CUDA	359.4	12.1	6.77	21.5	11.3	7.46
Ours	401.6	14.4	10.0	32.9	18.95	13.2
Speedup	1.12x	1.19x	1.48x	1.53x	1.68x	1.77x

4.6.1.5 Tensor Cache 策略测试

当启用了数据卸载/预取策略后，可以想象将会引入大量频繁的 GPU 内存申请和释放操作，而 `cudaMalloc` 和 `cudaFree` 在实践中较为低效，因此我们引入了 Tensor Cache，其内实现了基于 LRU 策略的缓存技术，避免了绝大部分不必要

的 CPU/GPU 内存搬移。为了测量其有效性，我们在开启数据卸载/预取策略后，在 AlexNet 网络上测试随着 batch-size 不断增长时的表现，同时在不同网络中测试 Tensor Cache 对训练速度的影响情况。

如表4-3所示，在未开启 Tensor Cache 时，所有的 checkpoints (CONV) 层在未被使用时会被卸载到 CPU 上，同时会在将来被预取到 GPU 上，带来了频繁的数据迁移。而随着 batch-size 增长，checkpoints 层输出的张量亦会增长，因此可以看到较为可观的数据迁移增长。而开启 Tensor Cache 后，在 256-896 batch size 时，因为此时 GPU 上空闲内存足够，由 LRU 原则可知，此时所有的 checkpoints 层将被缓存到了 GPU 上；而当 batch-size 增大到了 1024 后，此时 GPU 内存出现不足，因此会被 LRU 调度，把部分张量迁移到了 CPU 上。

虽然数据卸载/预取都是异步传输，然而其传输通道 PCI-e 通道速度远慢于 GPU 运算速度，因此仍然会带来不可避免的等待。因此我们测试了开启/关闭 Tensor Cache 后对训练速度的影响。如图4-13所示。在大型的线性结构网络如 AlexNet 和 VGG16 中，其计算密度远远高于非线性结构网络，计算时间大于传输耗时，因此 Tensor Cache 带来的提升并不明显；而在非线性结构网络中，计算密度较低，每一层的计算耗时极低，导致了计算时间小于传输时间，出现了等待现象，降低了训练速度。

表 4-3 Tensor Cache 对传输数据量的影响，测试网络为 AlexNet，开启张量生存期分析和数据卸载/预取策略。

Communications(GB)	256	384	512	640	896	1024
Without Tensor Cache	2.56	3.72	4.88	6.03	8.35	9.50
Tensor Cache	0	0	0	0	0	0.88

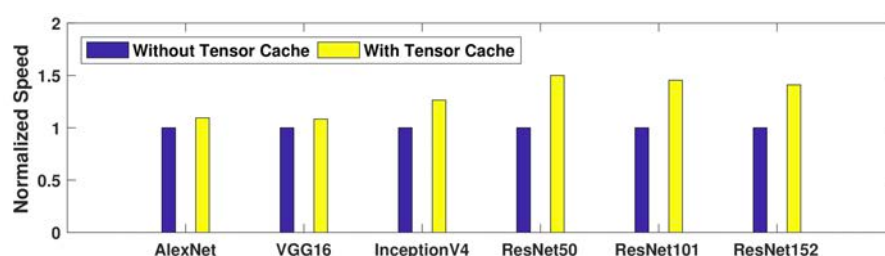


图 4-13 Tensor Cache 加速比，此时 AlexNet 网络的 batch-size 为 128，其余为 32。可见 Tensor Cache 在深层网络下优势明显。

4.6.2 综合测试

为了尽可能在单卡上训练跟大的模型，我们设计并实现了一系列的动态内存管理技术。接下来我们将在单个 NVIDIA Xp 卡（GPU 内存为 12GB）测试单卡能够承受模型的尺寸极限，以及与其他主流计算平台，如 Caffe^①，MxNet^②，Torch^③，及 TensorFlow^④的速度差异。

4.6.2.1 更深、更宽的网络训练

这里我们设计了两组实验，分别测试不同平台在同样的网络下能达到的最大的 batch-size，以及以同样 batch-size 情况下所能达到最深的网络。

第一组使用了 ResNet 进行实验。ResNet 结构分为 4 个不同的 blocks 类型，其深度由 blocks 的数量决定， $depth = 3 * (n_1 + n_2 + n_3 + n_4) + 2$ ，其中 n_i 表示第 i 个 blocks 的层数。我们固定了 ResNet 的 batch-size 为 16， $n_1 = 6, n_2 = 32, n_4 = 6$ ，通过改变 n_3 来改变网络的深度。表4-4展示了不同平台下 ResNet 结构所能达到的最大深度。可以发现，我们的实现所能达到的最大深度远超目前主流平台，约是 TensorFlow 的 3 倍之多。

表 4-4 不同平台下能达到的 ResNet 最大深度。batch-size 设置为 16。

Depth	Caffe	MxNet	Torch	TensorFlow	Ours
ResNet	148	480	152	592	1920

第二组实验是固定网络结构，逐渐增大 batch-size 到达平台所能训练的极限，对比不同平台在内存管理方面的差异。表4-5展示了不同平台在几个主流网络结构下所能训练的最大 batch-size。可以发现我们的平台能承受更大的 batch-size，在非线性结构网络方面尤其突出。其原因是在线性结构网络如 AlexNet 等，计算密度原高于非线性网络，平台的 overhead 普遍较小，动态内存管理所能发挥的作用有限。而在非线性结构网络方面，计算密度较低，且普遍较深，我们提出的如张量生存期分析、Tensor Cache 等技术在深层网络方面有着更大的优势。图4-14是根据上述的最大 batch-size 所绘制的网络内存总量。我们的实现在非线性网络方面效果显著，普遍超越 Caffe 10 倍以上，是 TensorFlow 的 3 倍以上。

① Caffe 1.0.0

② MxNet 1.0.0

③ Torch7, with cudnn-5 and enable cudnnAutoTune

④ TensorFlow 1.3.0

表 4-5 主流平台所能训练的模型最大 batch-size 值，在单块 NVIDIA Xp 卡上。

peak batch	Caffe	MxNet	Torch	TensorFlow	Ours
AlexNet	768	768	1024	1408	1792
VGG16	48	64	48	80	224
InceptionV4	16	N/A	N/A	64	240
ResNet50	32	80	32	128	384
ResNet101	16	48	16	80	256
ResNet152	16	32	16	48	176

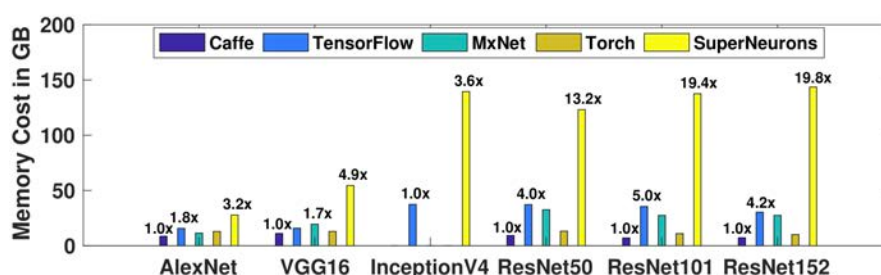


图 4-14 根据表4-5绘制的平台管理内存总量 $\sum_{i=1}^N (l_i^f + l_i^b)$ 。相比 Caffe，我们的实现能管理高达 19.8 倍的模型内存消耗。

4.6.2.2 速度测试

在我们提出的几项技术中，数据卸载/预取，再计算策略等对训练速度有较大影响，在极端情况下仅有单层网络的张量存活于 GPU 内存，每一层的计算都需从 CPU 搬移内存。而如动态卷积选择算法则对训练速度有利，为此我们综合了上述提出的所有技术，与现有主流平台对比测试在不同 batch-size 下的训练速度。图4-15展示了不同平台的速度对比测试。由于主流的平台在计算层面是基于 CUDNN 和 CUBLAS 代码包实现的，差异在于内存调度和管理方面，若平台能更好地管理和调度 GPU 内存，则能提供给卷积层更大的空闲内存，用于选择更高效的算法，结果是带来了更快的训练速度。而随着 batch-size 的不断增大，压缩了卷积层可用的空闲内存，导致训练速度逐步下降。

4.7 本章小结

本章尝试从工程实践的角度解决大型网络训练问题，即计算卡的内存资源有限，难以容纳足够大的模型，以致于需要依赖分布式训练等技术，引入了难以忍受的网络传输消耗。本章提出了几种动态内存管理方案，如张量生存期分析，统一的内存池技术，Tensor Cache 策略，代价最优的再计算策略等，使得更为大型

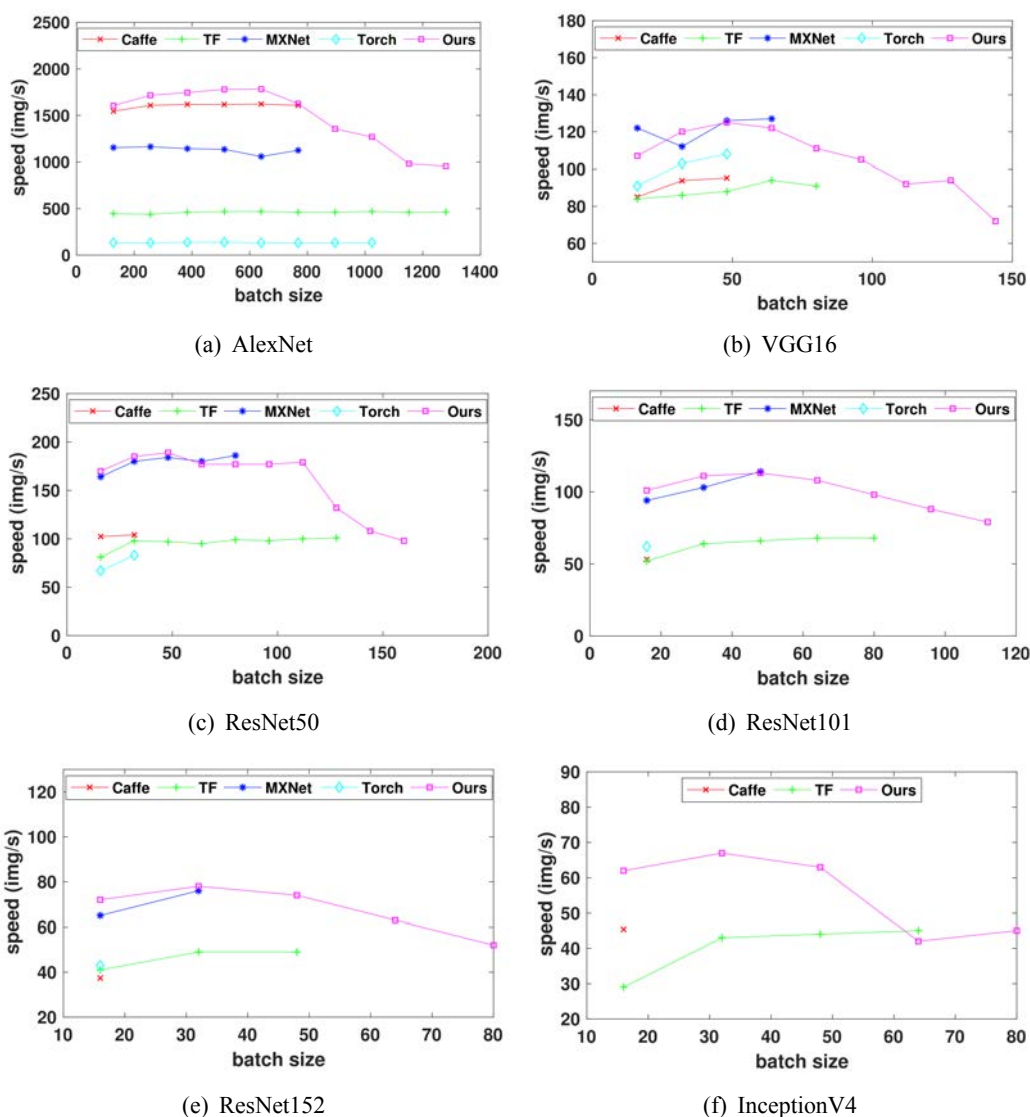


图 4-15 在 NVIDIA TITAN Xp 卡上的速度对比测试。

的模型训练成为可能。同时为了保持足够的训练速度，我们设计了动态卷积选择算法，以充分利用空闲内存，加速训练。我们完成了众多测试内容，包括每一项技术的单项测试，确保它们如预期工作；同时综合对比主流的计算平台，证明我们的技术的优越性能。近年来，在单卡训练方面还发展出了混合精度训练，利用 Tensor Core 进行，进一步推动了网络训练技术的进步。

第五章 全文总结与展望

5.1 全文总结

本文针对内存资源受限情况下的神经网络训练问题，分别从模型研究和工程实践的角度出发，提出了一个新的循环神经网络模型和一套动态内存管理方案，具体的内容有：

1. 本文针对循环神经网络在处理高维输入数据时输入层效率低下的问题，调研了现阶段神经网络模型裁剪的主流方法，以及紧凑循环神经网络最新的研究进展，发现目前已有的 Tensor-Train 分解模型在循环神经网络中应用的不足，由此引入了 Block-Term 块张量分解模型，重构了低效冗余的全连接层设计，提出了紧凑设计的 BT-LSTM 网络。在模型复杂度分析上，BT-LSTM 与 TT-LSTM 相当，而在实验中 BT-LSTM 在精度和收敛速度方面都优于 TT-LSTM 模型；同时以极低参数量构建的 BT-LSTM 网络获得同等甚至超越了传统 LSTM 网络的测试精度，尤其是在视频分类任务上效果突出。

2. 针对传统训练平台未能很好地处理非线性结构网络的变量依赖问题，和新一代训练平台在内存策略方面的不足，我们提出了三个动态内存管理技术：1) 张量生存期分析通过构建计算过程中每一步的 in/out 集合完成依赖分析，更精确管理内存的申请和释放时机，约能节省 40% 的内存使用；2) 逻辑统一的内存池实现方案，并辅以数据预取/卸载方案，以及 LRU 缓存策略，在以少量数据传输开销进一步节省约 10% 的内存开销；3) 最后我们提出了优化过的代价最优再计算策略，解决了 MxNet 等平台再计算策略开销过大的问题，以少量额外的计算开销，节省约 10% 的内存消耗。最后三者的有机结合，使得最终能节省超过 60% 的内存，并维持可观的训练速度。此外我们还加入了动态卷积算法选择策略，通过节省出更多的空闲内存来选择更快的卷积算法，加快网络训练速度。测试结果表明我们的策略可以使更宽更深的网络训练成为可能。

5.2 未来工作展望

本文所提方法在一定程度上缓解了内存资源受限的问题，但此问题将一直存在，未来仍有许多工作等待完成，这里列举一些在近期可行的工作展望：

1. Block-Term 模型是多个 Tucker 模型的线性组合，同时拥有 CP 分解和 Tucker 分解的优点。那么另一个可行的办法是多个 Tensor-Train 模型的线性组合，以此来弥补超参数设置不妥带来的问题，同时还拥有了 Tensor-Train 模型结构均衡的优

点。实际上 Tensor-Ring 分解算法^[93] 就可以看做是多个 Tensor-Train 的线性组合, 我们期待未来有更多关于 Tensor-Ring 应用例子的出现;

2. 无论是 Tensor-Train 模型还是 Block-Term 模型, 都存在未能有效降低计算复杂度的问题。近年来以多层次张量分解为理论基础的张量网络^[85] 开始出现在深度学习领域, 更有以全张量计算为设计基础的模型出现, 如文章^[111] 提出了一个二维依赖关系的卷积神经网络, 并借鉴了量子计算领域的 MERA 算法进行模型训练。我们期望一个全张量网络的出现, 一方面能够更高效处理高维数据输入的难题, 另一方面能够以更快的速度进行模型训练;

3. 现阶段的训练平台仍然处于不成熟的阶段, 尤其是在异构计算和分布式计算方面, 存在调度开销大、通信延迟影响严重的问题。从工程实践的角度难以获得长足的发展, 探索新的适合分布式计算的优化方法更为关键。基于随机梯度下降优化方法的模型, 每一步训练都需要对梯度进行一次 AllReduce 通信操作, 而此时计算核只能空转等待。那么是否可以引入异步或半异步计算的方式, 解除全局同步的开销, 同时保持与同步模型同样的训练精度。实际上近年来已经采用了部分梯度更新的策略来降低通信开销^[112], 说明全局梯度同步的限制其实是可以解除的。我们期待未来有更多关于分布式计算优化的方案出现, 进一步提高协处理器的计算效率, 避免不必要的等待开销;

4. GPU 最初被设计成为图像处理的协处理器, 它在架构设计方面并不是深度学习计算的最佳选项。此外, 由于 GPU 片上内存大小的限制和 PCI-e 通道速度的限制, 需要耗费大量人力来设计可行的资源调度方案。近年来出现了新一代的深度学习协处理器, 如 Google 的 TPU^[27] 等, 它们针对深度学习领域计算的特性来设计和制造, 能以极低的功耗达到更高的计算速度, 同时辅以更大的设备内存, 进一步加速了网络模型的训练和推理速度。我们期待未来有更廉价的高效协处理器, 加速深度学习领域在全社会的推广, 推动人类社会的技术革新浪潮。

致 谢

在读硕期间，非常感谢我的导师徐增林教授，带我入门并领略到了机器学习和深度学习的风采，让我得以从另一个新的角度观看世界。导师严谨的治学风格让我受益匪浅，让我明白了无论是学习还是工作，都需一丝不苟，兢兢业业。作为一名“青千”教授，更是实验室的负责人，他忙碌的背影给我很大的触动，也越发明白那光鲜的头衔背后曾经流下的辛酸汗水。非常感谢人生路上能有这么一位名师，祝愿导师著作等身高，桃李满天下。

其次要感谢香港中文大学 Irwin King 和 Michael R. Lyu 两位教授，感谢他们给予我宝贵的去香港交流的机会，这对开阔我的眼界，强化自身能力方面起到了极大的帮助作用。同时感谢 360 公司的陈强研究员给予我宝贵的实习机会，尤其感谢 360 公司的颜水成院长，即使诸事烦身还能坚持每周与我进行会议并讨论研究进展。

其次感谢 SMILE Lab 的所有师生，感谢他们共同营造了温馨自由的科研环境，能让我心无旁骛地学习、工作和游玩，度过了三年愉快的硕士时光。尤其是要感谢李广西同学，感谢他在学习上给予我的引导和帮助。此外还要感谢陈迪同学和罗聪健同学，感谢他们一同帮忙完成实验室的集群管理。

最后要将诚挚的谢意献给我的家人，感谢他们一直以来对我的鼓励和支持。

参考文献

- [1] S. Ren, K. He, R. B. Girshick, et al. Faster R-CNN: towards real-time object detection with region proposal networks[C]. Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada, 2015, 91-99
- [2] R. B. Girshick. Fast R-CNN[C]. 2015 IEEE International Conference on Computer Vision, ICCV 2015, Santiago, Chile, December 7-13, 2015, 2015, 1440-1448
- [3] K. He, G. Gkioxari, P. Dollár, et al. Mask R-CNN[C]. IEEE International Conference on Computer Vision, ICCV 2017, Venice, Italy, October 22-29, 2017, 2017, 2980-2988
- [4] O. Vinyals, A. Toshev, S. Bengio, et al. Show and tell: A neural image caption generator[C]. IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015, 2015, 3156-3164
- [5] J. Lu, C. Xiong, D. Parikh, et al. Knowing when to look: Adaptive attention via a visual sentinel for image captioning[C]. 2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, Honolulu, HI, USA, July 21-26, 2017, 2017, 3242-3250
- [6] A. van den Oord, S. Dieleman, H. Zen, et al. Wavenet: A generative model for raw audio[C]. The 9th ISCA Speech Synthesis Workshop, Sunnyvale, CA, USA, 13-15 September 2016, 2016, 125
- [7] A. Vaswani, N. Shazeer, N. Parmar, et al. Attention is all you need[C]. Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, 4-9 December 2017, Long Beach, CA, USA, 2017, 6000-6010
- [8] M. Zitnik, M. Agrawal, J. Leskovec. Modeling polypharmacy side effects with graph convolutional networks[J]. Bioinformatics, 2018, 34(13): i457-i466
- [9] J. Zhou, G. Cui, Z. Zhang, et al. Graph neural networks: A review of methods and applications[J]. CoRR, 2018, abs/1812.08434: 1-20
- [10] K. Fukushima. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position[J]. Biological cybernetics, 1980, 36(4): 193-202
- [11] Y. LeCun, Y. Bengio, et al. Convolutional networks for images, speech, and time series[J]. The handbook of brain theory and neural networks, 1995, 3361(10): 1-14
- [12] D. E. Rumelhart, G. E. Hinton, R. J. Williams. Learning representations by back-propagating errors[J]. Nature, 1986, 323: 533-536

- [13] G. E. Hinton, J. L. McClelland. Learning representations by recirculation[C]. Neural Information Processing Systems, Denver, Colorado, USA, 1987, 1987, 358-366
- [14] Y. Li, H. Ai, T. Yamashita, et al. Tracking in low frame rate video: A cascade particle filter with discriminative observers of different life spans[J]. IEEE Trans. Pattern Anal. Mach. Intell., 2008, 30(10): 1728-1740
- [15] X. Cao, Y. Wei, F. Wen, et al. Face alignment by explicit shape regression[J]. International Journal of Computer Vision, 2014, 107(2): 177-190
- [16] K. Bengler, K. Dietmayer, B. Färber, et al. Three decades of driver assistance systems: Review and future perspectives[J]. IEEE Intell. Transport. Syst. Mag., 2014, 6(4): 6-22
- [17] Y. Zhou, O. Tuzel. Voxynet: End-to-end learning for point cloud based 3d object detection[C]. 2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018, 2018, 4490-4499
- [18] O. Ronneberger, P. Fischer, T. Brox. U-net: Convolutional networks for biomedical image segmentation[C]. Medical Image Computing and Computer-Assisted Intervention - MICCAI 2015 - 18th International Conference Munich, Germany, October 5 - 9, 2015, Proceedings, Part III, 2015, 234-241
- [19] M. Liu, J. Zhang, E. Adeli, et al. Deep multi-task multi-channel learning for joint classification and regression of brain status[C]. Medical Image Computing and Computer Assisted Intervention - MICCAI 2017 - 20th International Conference, Quebec City, QC, Canada, September 11-13, 2017, Proceedings, Part III, 2017, 3-11
- [20] T. Mikolov, I. Sutskever, K. Chen, et al. Distributed representations of words and phrases and their compositionality[C]. Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013. Proceedings of a meeting held December 5-8, 2013, Lake Tahoe, Nevada, United States., 2013, 3111-3119
- [21] I. Sutskever, O. Vinyals, Q. V. Le. Sequence to sequence learning with neural networks[C]. Advances in Neural Information Processing Systems 27: Annual Conference on Neural Information Processing Systems 2014, December 8-13 2014, Montreal, Quebec, Canada, 2014, 3104-3112
- [22] B. Hidasi, A. Karatzoglou, L. Baltrunas, et al. Session-based recommendations with recurrent neural networks[J]. CoRR, 2015, abs/1511.06939: 1-10

- [23] S. Zhai, K. Chang, R. Zhang, et al. Deepintent: Learning attentions for online advertising with recurrent neural networks[C]. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13-17, 2016, 2016, 1295-1304
- [24] Y. Du, W. Wang, L. Wang. Hierarchical recurrent neural network for skeleton based action recognition[C]. IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015, 2015, 1110-1118
- [25] Q. Gan, Q. Guo, Z. Zhang, et al. First step toward model-free, anonymous object tracking with recurrent neural networks[J]. CoRR, 2015, abs/1511.06425: 1-12
- [26] J. Devlin, M. Chang, K. Lee, et al. BERT: pre-training of deep bidirectional transformers for language understanding[J]. CoRR, 2018, abs/1810.04805: 1-12
- [27] N. P. Jouppi, C. Young, N. Patil, et al. In-datacenter performance analysis of a tensor processing unit[C]. Proceedings of the 44th Annual International Symposium on Computer Architecture, ISCA 2017, Toronto, ON, Canada, June 24-28, 2017, 2017, 1-12
- [28] L. Wang, J. Ye, Y. Zhao, et al. Superneurons: dynamic GPU memory management for training deep neural networks[C]. Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP 2018, Vienna, Austria, February 24-28, 2018, 2018, 41-53
- [29] Y. LeCun, L. Bottou, Y. Bengio, et al. Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998, 86(11): 2278-2324
- [30] D. H. Hubel, T. N. Wiesel. Receptive fields and functional architecture of monkey striate cortex.[J]. The Journal of physiology, 1968, 195 1: 215-43
- [31] I. J. Goodfellow, Y. Bengio, A. C. Courville. Deep learning[M]. MIT Press, 2016
- [32] S. Liang, R. Srikant. Why deep neural networks?[J]. CoRR, 2016, abs/1610.04161: 1-17
- [33] A. Krizhevsky, I. Sutskever, G. E. Hinton. Imagenet classification with deep convolutional neural networks[C]. Advances in Neural Information Processing Systems 25: 26th Annual Conference on Neural Information Processing Systems 2012. Proceedings of a meeting held December 3-6, 2012, Lake Tahoe, Nevada, United States., 2012, 1106-1114
- [34] K. Simonyan, A. Zisserman. Very deep convolutional networks for large-scale image recognition[J]. CoRR, 2014, abs/1409.1556: 1-15
- [35] N. Srivastava, G. E. Hinton, A. Krizhevsky, et al. Dropout: a simple way to prevent neural networks from overfitting[J]. Journal of Machine Learning Research, 2014, 15(1): 1929-1958

- [36] S. Ioffe, C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift[C]. Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015, 2015, 448-456
- [37] K. He, X. Zhang, S. Ren, et al. Deep residual learning for image recognition[C]. 2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016, 2016, 770-778
- [38] A. G. Howard, M. Zhu, B. Chen, et al. Mobilenets: Efficient convolutional neural networks for mobile vision applications[J]. CoRR, 2017, abs/1704.04861: 1-12
- [39] I. Hubara, M. Courbariaux, D. Soudry, et al. Binarized neural networks[C]. Advances in Neural Information Processing Systems 29: Annual Conference on Neural Information Processing Systems 2016, December 5-10, 2016, Barcelona, Spain, 2016, 4107-4115
- [40] S. Gupta, A. Agrawal, K. Gopalakrishnan, et al. Deep learning with limited numerical precision[C]. Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015, 2015, 1737-1746
- [41] V. Vanhoucke, A. Senior, M. Z. Mao. Improving the speed of neural networks on cpus[C]. Proc. Deep Learning and Unsupervised Feature Learning NIPS Workshop, 2011, 1-4
- [42] M. Rastegari, V. Ordonez, J. Redmon, et al. Xnor-net: Imagenet classification using binary convolutional neural networks[C]. Computer Vision - ECCV 2016 - 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part IV, 2016, 525-542
- [43] M. Courbariaux, Y. Bengio, J. David. Binaryconnect: Training deep neural networks with binary weights during propagations[C]. Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada, 2015, 3123-3131
- [44] J. Ott, Z. Lin, Y. Zhang, et al. Recurrent neural networks with limited numerical precision[J]. CoRR, 2016, abs/1611.07065: 1-9
- [45] C. Zhu, S. Han, H. Mao, et al. Trained ternary quantization[J]. CoRR, 2016, abs/1612.01064: 1-8
- [46] S. Zhou, Z. Ni, X. Zhou, et al. Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients[J]. CoRR, 2016, abs/1606.06160: 1-11
- [47] W. Wen, C. Xu, F. Yan, et al. Terngrad: Ternary gradients to reduce communication in distributed deep learning[C]. Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, 4-9 December 2017, Long Beach, CA, USA, 2017, 1508-1518

- [48] S. Wu, G. Li, F. Chen, et al. Training and inference with integers in deep neural networks[J]. CoRR, 2018, abs/1802.04680: 1-12
- [49] J. Ba, R. Caruana. Do deep nets really need to be deep?[C]. Advances in Neural Information Processing Systems 27: Annual Conference on Neural Information Processing Systems 2014, December 8-13 2014, Montreal, Quebec, Canada, 2014, 2654-2662
- [50] G. E. Hinton, O. Vinyals, J. Dean. Distilling the knowledge in a neural network[J]. CoRR, 2015, abs/1503.02531: 1-14
- [51] A. K. Balan, V. Rathod, K. P. Murphy, et al. Bayesian dark knowledge[C]. Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada, 2015, 3438-3446
- [52] I. Radosavovic, P. Dollár, R. B. Girshick, et al. Data distillation: Towards omni-supervised learning[C]. 2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018, 2018, 4119-4128
- [53] T. He, Z. Zhang, H. Zhang, et al. Bag of tricks for image classification with convolutional neural networks[J]. CoRR, 2018, abs/1812.01187: 1-10
- [54] W. Wen, C. Wu, Y. Wang, et al. Learning structured sparsity in deep neural networks[C]. Advances in Neural Information Processing Systems 29: Annual Conference on Neural Information Processing Systems 2016, December 5-10, 2016, Barcelona, Spain, 2016, 2074-2082
- [55] W. Wen, Y. He, S. Rajbhandari, et al. Learning intrinsic sparse structures within long short-term memory[J]. CoRR, 2017, abs/1709.05027: 1-12
- [56] S. Narang, G. F. Damos, S. Sengupta, et al. Exploring sparsity in recurrent neural networks[J]. CoRR, 2017, abs/1704.05119: 1-10
- [57] Z. Yang, M. Moczulski, M. Denil, et al. Deep fried convnets[C]. 2015 IEEE International Conference on Computer Vision, ICCV 2015, Santiago, Chile, December 7-13, 2015, 2015, 1476-1483
- [58] T. N. Sainath, B. Kingsbury, V. Sindhwani, et al. Low-rank matrix factorization for deep neural network training with high-dimensional output targets[C]. IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2013, Vancouver, BC, Canada, May 26-31, 2013, 2013, 6655-6659
- [59] M. Denil, B. Shakibi, L. Dinh, et al. Predicting parameters in deep learning[C]. Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013. Proceedings of a meeting held December 5-8, 2013, Lake Tahoe, Nevada, United States., 2013, 2148-2156

- [60] J. Xue, J. Li, Y. Gong. Restructuring of deep neural network acoustic models with singular value decomposition[C]. INTERSPEECH 2013, 14th Annual Conference of the International Speech Communication Association, Lyon, France, August 25-29, 2013, 2013, 2365-2369
- [61] T. G. Kolda, B. W. Bader. Tensor decompositions and applications[J]. SIAM Review, 2009, 51(3): 455-500
- [62] A. Novikov, D. Podoprikin, A. Osokin, et al. Tensorizing neural networks[C]. Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada, 2015, 442-450
- [63] Z. He, S. Gao, L. Xiao, et al. Wider and deeper, cheaper and faster: Tensorized lstms for sequence learning[C]. Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, 4-9 December 2017, Long Beach, CA, USA, 2017, 1-11
- [64] I. V. Oseledets. Tensor-train decomposition[J]. SIAM J. Scientific Computing, 2011, 33(5): 2295-2317
- [65] Y. Yang, D. Krompass, V. Tresp. Tensor-train recurrent neural networks for video classification[C]. Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017, 2017, 3891-3900
- [66] R. Yu, S. Zheng, A. Anandkumar, et al. Long-term forecasting using tensor-train rnns[J]. CoRR, 2017, abs/1711.00073: 1-9
- [67] N. Tishby, F. C. N. Pereira, W. Bialek. The information bottleneck method[J]. CoRR, 2000, physics/0004057: 1-8
- [68] J. Ye, L. Wang, G. Li, et al. Learning compact recurrent neural networks with block-term tensor decomposition[C]. 2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018, 2018, 9378-9387
- [69] L. D. Lathauwer. Decompositions of a higher-order tensor in block terms - part II: definitions and uniqueness[J]. SIAM J. Matrix Analysis Applications, 2008, 30(3): 1033-1066
- [70] C. Szegedy, S. Ioffe, V. Vanhoucke, et al. Inception-v4, inception-resnet and the impact of residual connections on learning[C]. Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence, February 4-9, 2017, San Francisco, California, USA., 2017, 4278-4284
- [71] S. Cook. Cuda 并程序序设计 [M]. 机械工业出版社, 2013

- [72] M. Abadi, P. Barham, J. Chen, et al. Tensorflow: A system for large-scale machine learning[C]. 12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016., 2016, 265-283
- [73] R. Collobert, S. Bengio, J. Mariéthoz. Torch: a modular machine learning software library[R]. Idiap, 2002
- [74] Y. Jia, E. Shelhamer, J. Donahue, et al. Caffé: Convolutional architecture for fast feature embedding[C]. Proceedings of the ACM International Conference on Multimedia, MM '14, Orlando, FL, USA, November 03 - 07, 2014, 2014, 675-678
- [75] T. Chen, M. Li, Y. Li, et al. Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems[J]. CoRR, 2015, abs/1512.01274: 1-8
- [76] A. Paszke, S. Gross, A. Lerer. Automatic differentiation in pytorch[C]. NIPS 2017 Workshop Autodiff, 2017, 1-8
- [77] 李航. 统计学习方法 [M]. 清华大学出版社, 2012
- [78] 周志华. 机器学习 [M]. 清华大学出版社, 2016
- [79] P. Goyal, P. Dollár, R. B. Girshick, et al. Accurate, large minibatch SGD: training imagenet in 1 hour[J]. CoRR, 2017, abs/1706.02677: 1-9
- [80] S. Chetlur, C. Woolley, P. Vandermersch, et al. cudnn: Efficient primitives for deep learning[J]. CoRR, 2014, abs/1410.0759: 1-7
- [81] S. Hochreiter, J. Schmidhuber. Long short-term memory[J]. Neural Computation, 1997, 9(8): 1735-1780
- [82] K. Cho, B. van Merriënboer, Ç. Gülçehre, et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation[C]. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL, 2014, 1724-1734
- [83] P. J. Werbos. Generalization of backpropagation with application to a recurrent gas market model[J]. Neural Networks, 1988, 1(4): 339-356
- [84] M. C. Mozer. A focused backpropagation algorithm for temporal pattern recognition[J]. Complex Systems, 1989, 3(4): 349-381
- [85] A. Cichocki. Era of big data processing: A new approach via tensor networks and tensor decompositions[J]. CoRR, 2014, abs/1403.2048: 1-10
- [86] 张贤达. 矩阵分析与应用 [M]. 清华大学出版社, 2004

- [87] J. D. Carroll, J.-J. Chang. Analysis of individual differences in multidimensional scaling via an n-way generalization of “eckart-young” decomposition[J]. Psychometrika, 1970, 35(3): 283-319
- [88] R. A. Harshman, et al. Foundations of the parafac procedure: Models and conditions for an “explanatory” multimodal factor analysis[J]. UCLA working papers in phonetics, 1970, 16(1970): 1-84
- [89] T. G. Kolda. Multilinear operators for higher-order decompositions.[R]. Sandia National Laboratories, 2006
- [90] D. P. Kingma, J. Ba. Adam: A method for stochastic optimization[J]. CoRR, 2014, abs/1412.6980: 1-12
- [91] N. Qian. On the momentum term in gradient descent learning algorithms[J]. Neural Networks, 1999, 12(1): 145-151
- [92] V. de Silva, L. Lim. Tensor rank and the ill-posedness of the best low-rank approximation problem[J]. SIAM J. Matrix Analysis Applications, 2008, 30(3): 1084-1127
- [93] Q. Zhao, M. Sugiyama, A. Cichocki. Learning efficient tensor representations with ring structure networks[J]. CoRR, 2017, abs/1705.08286: 1-12
- [94] L. R. Tucker. Implications of factor analysis of three-way matrices for measurement of change[J]. Problems in measuring change, 1963, 15: 122-137
- [95] L. D. Lathauwer, B. D. Moor, J. Vandewalle. A multilinear singular value decomposition[J]. SIAM J. Matrix Analysis Applications, 2000, 21(4): 1253-1278
- [96] D. Pérez-García, F. Verstraete, M. M. Wolf, et al. Matrix product state representations[J]. Quantum Information & Computation, 2007, 7(5): 401-430
- [97] N. Tishby, N. Zaslavsky. Deep learning and the information bottleneck principle[C]. 2015 IEEE Information Theory Workshop, ITW 2015, Jerusalem, Israel, April 26 - May 1, 2015, 2015, 1-5
- [98] X. Glorot, A. Bordes, Y. Bengio. Deep sparse rectifier neural networks[C]. Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, AISTATS 2011, Fort Lauderdale, USA, April 11-13, 2011, 2011, 315-323
- [99] C. Szegedy, W. Liu, Y. Jia, et al. Going deeper with convolutions[C]. IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015, 2015, 1-9
- [100] R. K. Srivastava, K. Greff, J. Schmidhuber. Highway networks[J]. CoRR, 2015, abs/1505.00387: 1-12

- [101] G. Huang, Z. Liu, K. Q. Weinberger. Densely connected convolutional networks[J]. CoRR, 2016, abs/1608.06993: 1-10
- [102] Y. Nesterov. A method for unconstrained convex minimization problem with the rate of convergence $O(1/k^2)$ [C]. Doklady AN USSR, 1983, 543-547
- [103] S. Ruder. An overview of gradient descent optimization algorithms[J]. CoRR, 2016, abs/1609.04747: 1-9
- [104] J. Liu, , M. Shah. Recognizing realistic actions from videos “in the wild”[C]. 2009 IEEE Conference on Computer Vision and Pattern Recognition, 2009, 1996-2003
- [105] S. Sharma, R. Kiros, R. Salakhutdinov. Action recognition using visual attention[J]. CoRR, 2015, abs/1511.04119: 1-10
- [106] D. Liu, M. Shyu, G. Zhao. Spatial-temporal motion information integration for action detection and recognition in non-static background[C]. IEEE 14th International Conference on Information Reuse & Integration, IRI 2013, San Francisco, CA, USA, August 14-16, 2013, 2013, 626-633
- [107] T. Lin, M. Maire, S. J. Belongie, et al. Microsoft COCO: common objects in context[C]. Computer Vision - ECCV 2014 - 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V, 2014, 740-755
- [108] K. Papineni, S. Roukos, T. Ward, et al. Bleu: a method for automatic evaluation of machine translation[C]. Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, July 6-12, 2002, Philadelphia, PA, USA., 2002, 311-318
- [109] R. Vedantam, C. L. Zitnick, D. Parikh. Cider: Consensus-based image description evaluation[J]. CoRR, 2014, abs/1411.5726: 1-11
- [110] T. Chen, B. Xu, C. Zhang, et al. Training deep nets with sublinear memory cost[J]. CoRR, 2016, abs/1604.06174: 1-10
- [111] D. Liu, S.-J. Ran, P. Wittek, et al. Machine Learning by Unitary Tensor Network of Hierarchical Tree Structure[J]. CoRR, 2017, abs/1710.04833: 1-10
- [112] P. Sun, W. Feng, R. Han, et al. Optimizing Network Performance for Distributed DNN Training on GPU Clusters: ImageNet/AlexNet Training in 1.5 Minutes[J]. CoRR, 2019, abs/1902.06855: 1-10
- [113] O. Russakovsky, J. Deng, H. Su, et al. Imagenet large scale visual recognition challenge[J]. International Journal of Computer Vision, 2015, 115(3): 211-252

- [114] Y. Cheng, F. X. Yu, R. S. Feris, et al. An exploration of parameter redundancy in deep networks with circulant projections[C]. 2015 IEEE International Conference on Computer Vision, ICCV 2015, Santiago, Chile, December 7-13, 2015, 2015, 2857-2865
- [115] Y. Cheng, F. X. Yu, R. S. Feris, et al. Fast neural networks with circulant projections[J]. CoRR, 2015, abs/1502.03436: 1-9
- [116] D. Liu, S.-J. Ran, P. Wittek, et al. Machine learning by two-dimensional hierarchical tensor networks: A quantum information theoretic perspective on deep architectures[J]. CoRR, 2017, abs/1710.04833: 1-8
- [117] Y. Pan, J. Xu, M. Wang, et al. Compressing recurrent neural networks with tensor ring for action recognition[J]. CoRR, 2018, abs/1811.07503: 1-8

攻读硕士学位期间取得的成果

- [1] **Jinmian Ye**, L. Wang, G. Li, et al. Learning compact recurrent neural networks with block-term tensor decomposition[C]. 2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018, 2018, 9378-9387
- [2] L. Wang, **Jinmian Ye**, Y. Zhao, et al. Superneurons: dynamic GPU memory management for training deep neural networks[C]. Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPoPP 2018, Vienna, Austria, February 24-28, 2018, 2018, 41-53
- [3] L. Hao, S. Liang, **Jinmian Ye**, et al. Tensord: A tensor decomposition library in tensorflow[J]. Neurocomputing, 2018, 318: 196-200
- [4] 徐增林, **叶锦棉**, 李广西, et al. 基于块张量分解的循环神经网络稀疏连接方法 [P]. 中国, 发明专利, CN201711290493, 2017 年 12 月 08 日
- [5] 徐增林, 李广西, **叶锦棉**, et al. 基于块项张量分解的深度神经网络压缩方法 [P]. 中国, 发明专利, CN201711319853, 2017 年 12 月 12 日